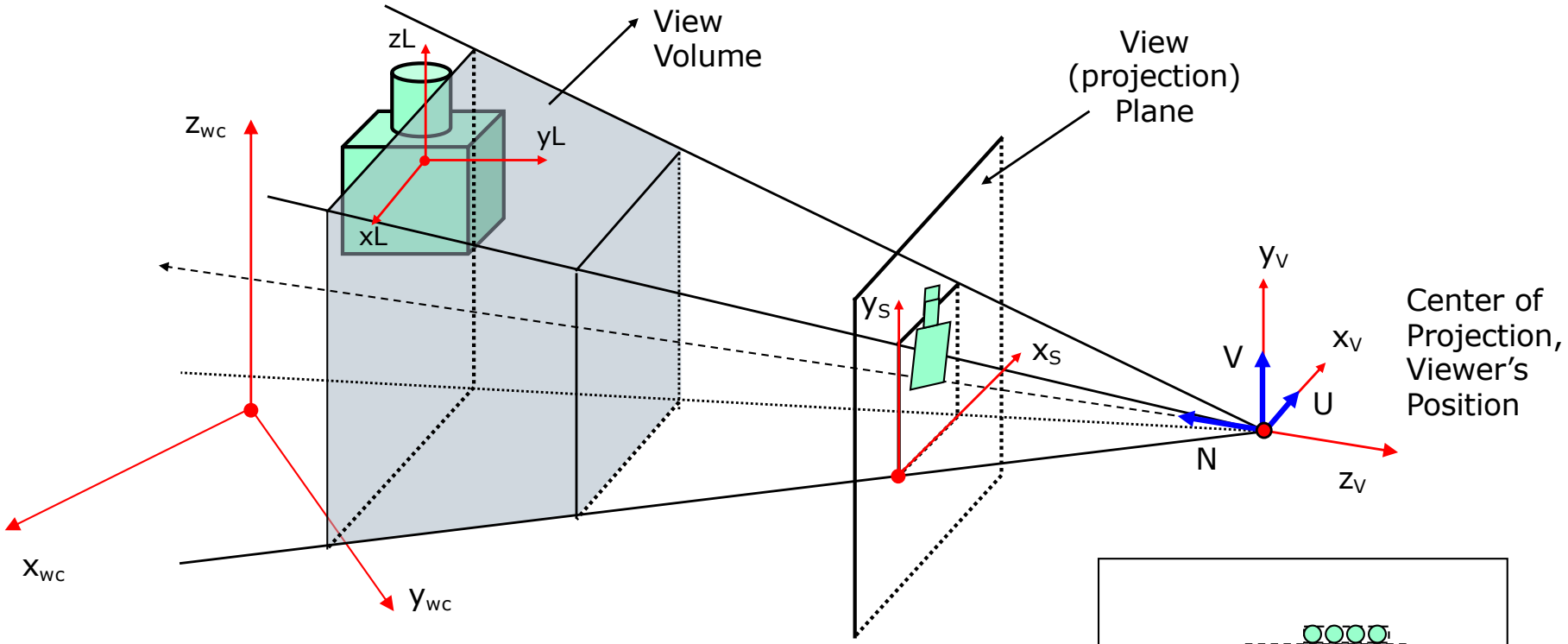
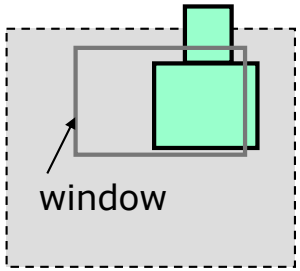
Hidden line and surface removal algorithms (2)

Contents

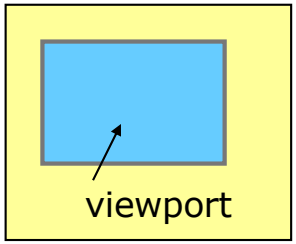
- Introduction
- Hidden line and surface removal algorithms
  1. Floating horizon algorithm
  2. Back-face culling
  3. Area subdivision algorithm
  4. Depth buffer algorithms
  5. **Depth sorting algorithms**
  6. **Scan-line algorithms**
  7. **Octree algorithm**
  8. **Ray casting algorithms**
  9. **BSP trees**

Conceptual model of 3D viewing process

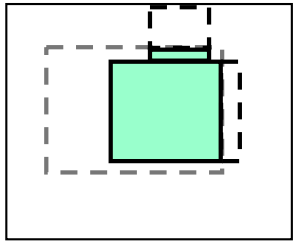




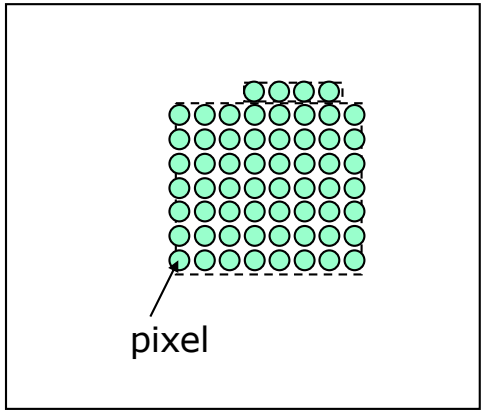
Window definition
in projection plane



Viewport definition
in projection plane



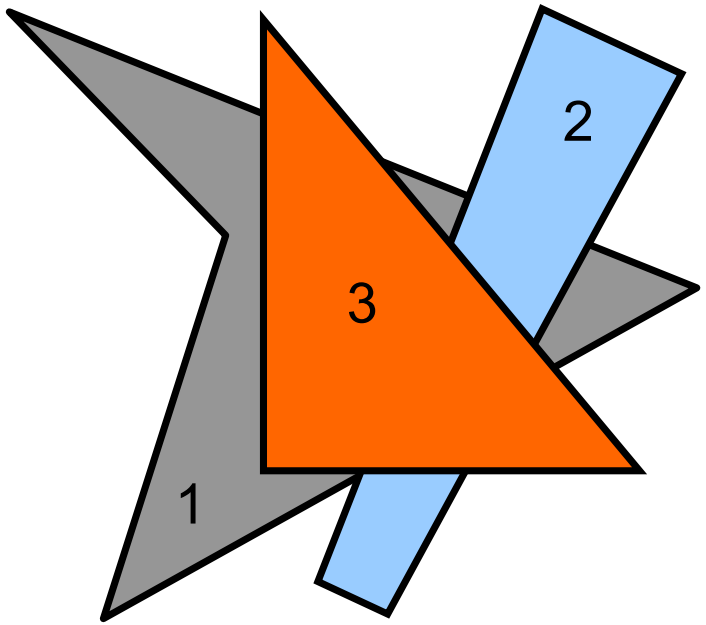
Window's content
inside the viewport



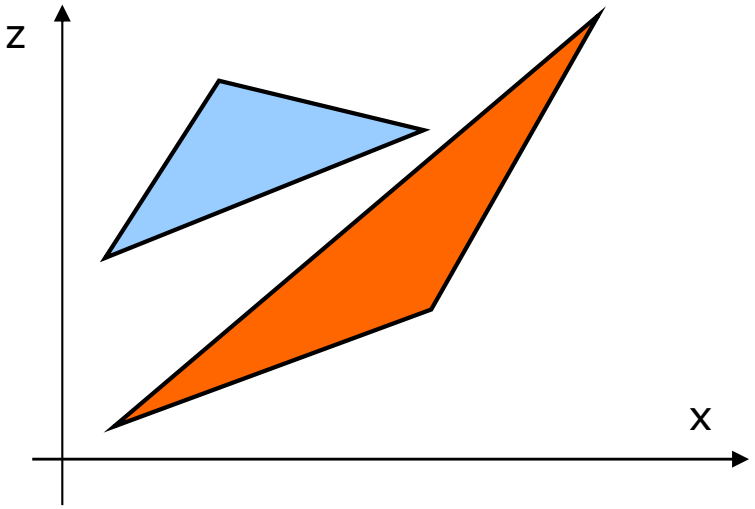
Scan conversion onto the
raster screen

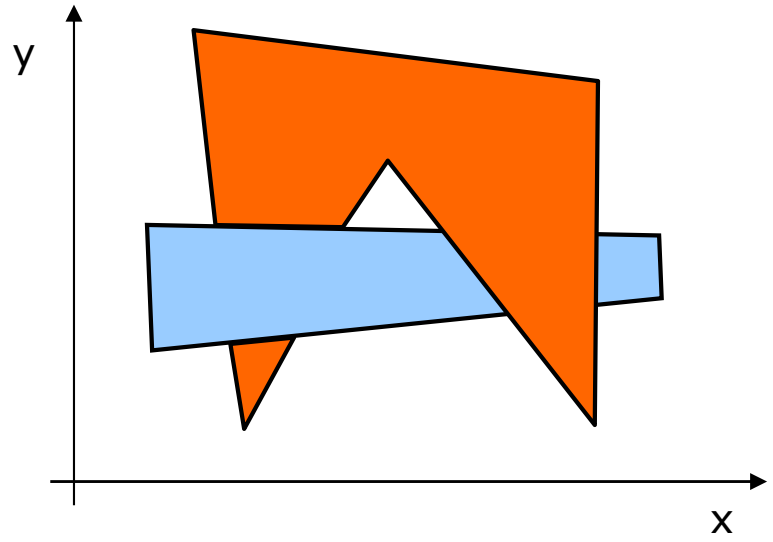
5. Depth Sorting Algorithm

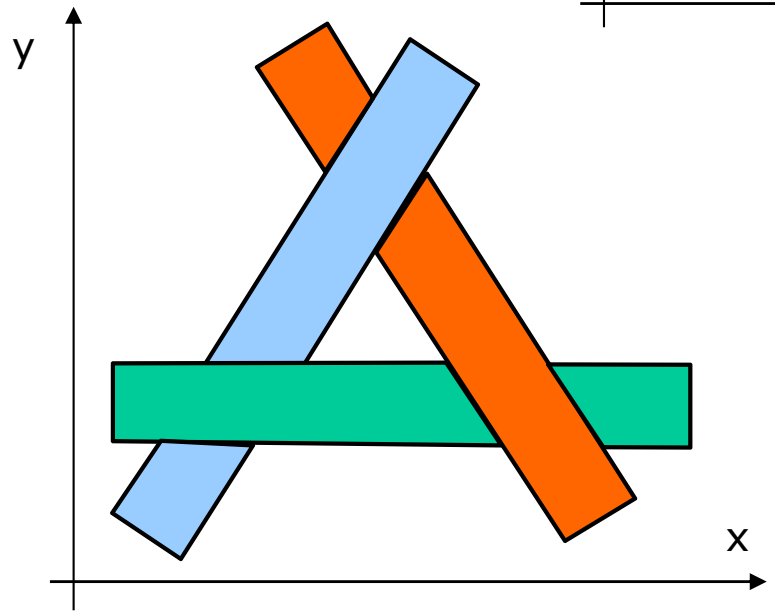
- Published in 1972, by Newell, M.E., Newell, R.G., Sancha, T.L., "A Solution to the Hidden Surface Problem", Proceedings of the ACM National Conference.
- Known as painter's algorithm
  - paint the polygons into the frame buffer in order of decreasing distance from the viewpoint.
- The algorithm consists of three steps:
  1. Sort all polygons according to the greatest (furthest) z coordinate of each
  2. Resolve any ambiguities that may cause when the polygons' z extents overlap, splitting polygons if necessary
  3. Scan convert each polygon in descending order of greatest z coordinate (i.e., back to front).



Cases of ambiguities



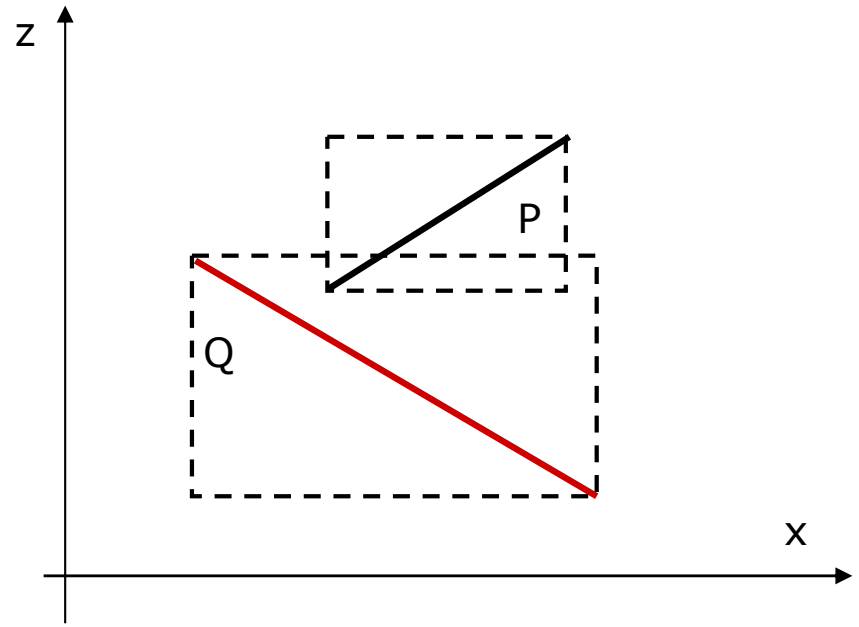




Depth sorting algorithm – 5 tests

- Let us consider the polygon P at the far end of the sorted list
- Before scan converting P into the frame buffer, it is tested against each polygon Q whose z extent overlaps the z extent of P. The test proves P cannot obscure Q and that P can be written in the list before Q.
- There are 5 tests in order of increasing complexity:
  1. Polygons' x extents not overlap.
  2. Polygons' y extents not overlap.
  3. Entire P on the opposite side of Q's plane from the viewpoint.
  4. Entire Q on the same side of P's plane as the viewpoint.
  5. Projections of the polygons onto the (x,y) plane not overlap.
- If all 5 tests fail, it can be assumed for the moment that P actually obscures Q, and therefore test whether Q can be scan-converted before P.
- The tests 3 and 4 are performed, with the polygons reversed:
  - 3* Entire Q on the opposite side of P's plane from the viewpoint
  - 4* Entire P on the same side of Q's plane as the viewpoint
- If the tests 3* and 4* succeed the polygon Q is moved to the end of the list and it becomes the new P. The Q polygon is marked to avoid the infinitely swapping.
- If the tests 3) and 4) fail again it means that either P or Q must be split by the plane of the other. The unsplit polygon is discharged, and its pieces are inserted into the list in proper z order, and the algorithm proceeds as before.

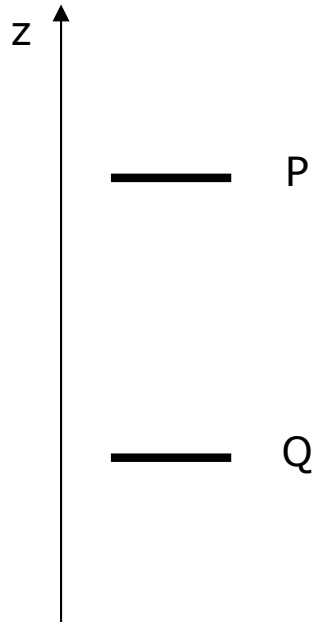
Checking normal case (1-4)



Test: P against Q

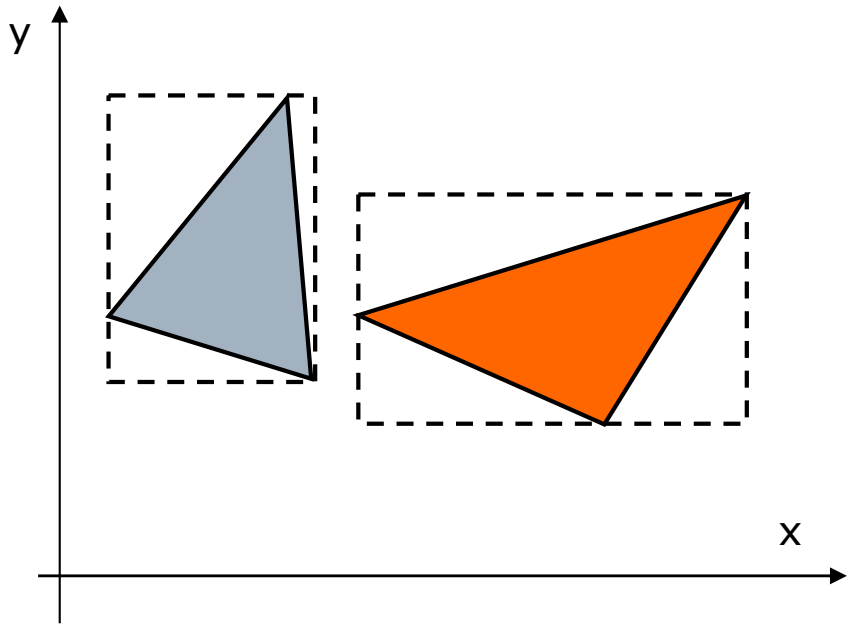
Viewing direction

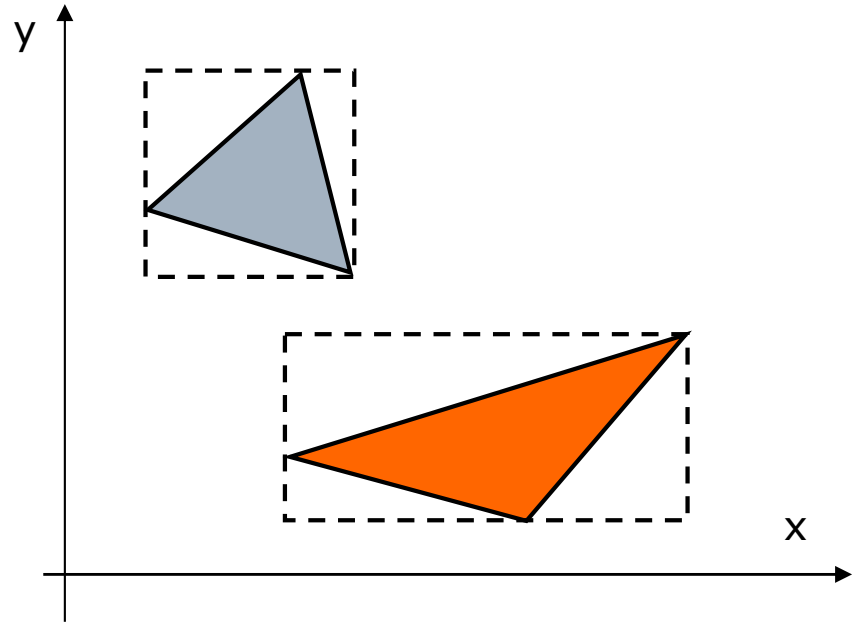
Case: 1,2,3, 4



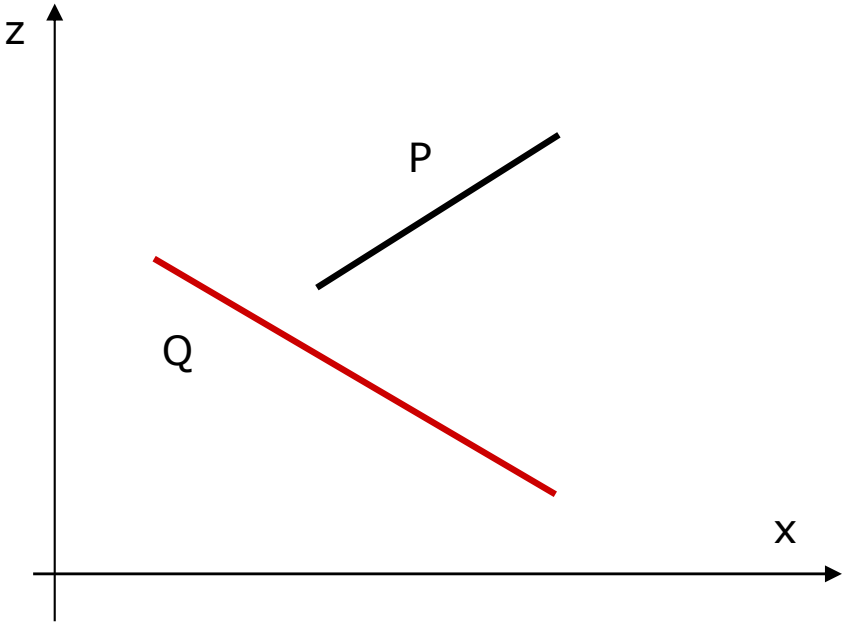
Viewing direction

Test 1 and 2

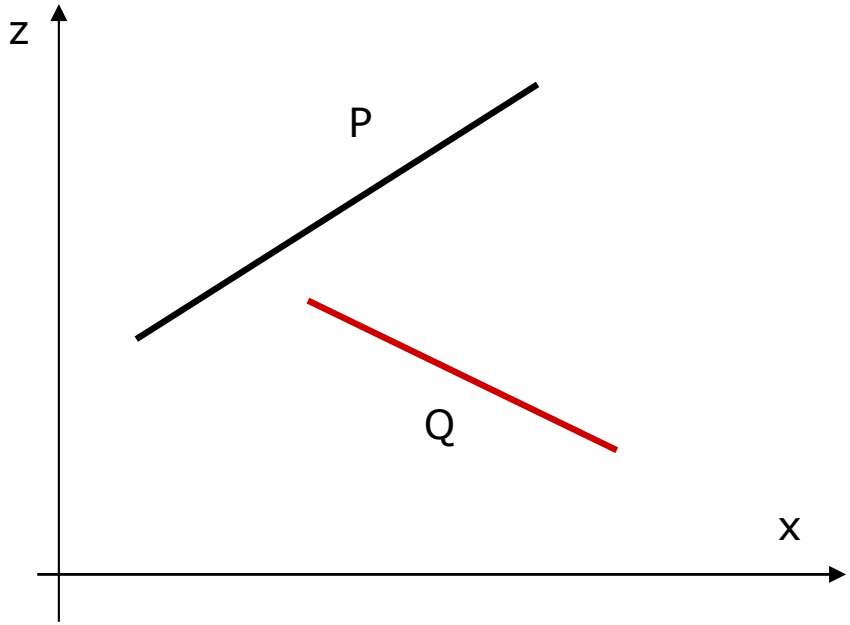




Test 3 and 4



Test 3: draw P, Q

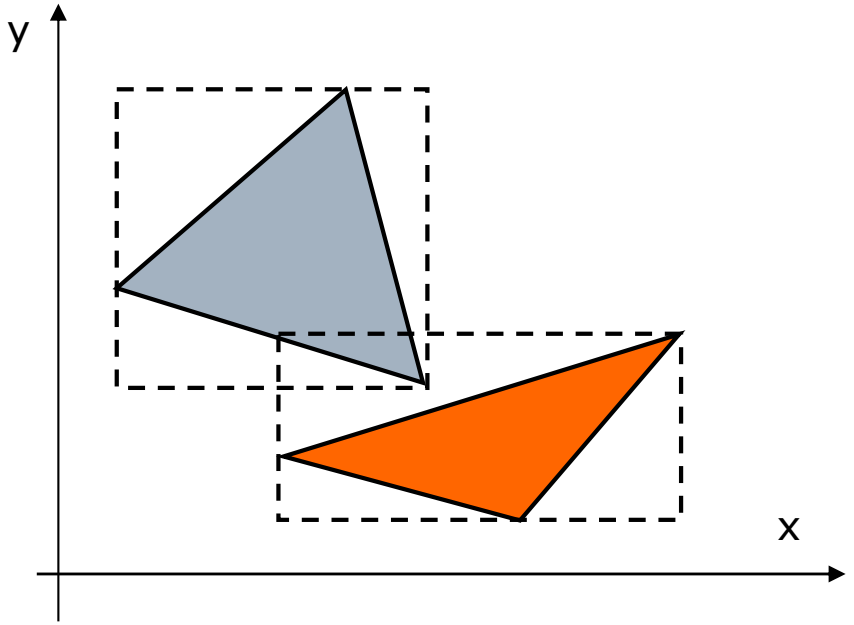


Test 4: draw P, Q

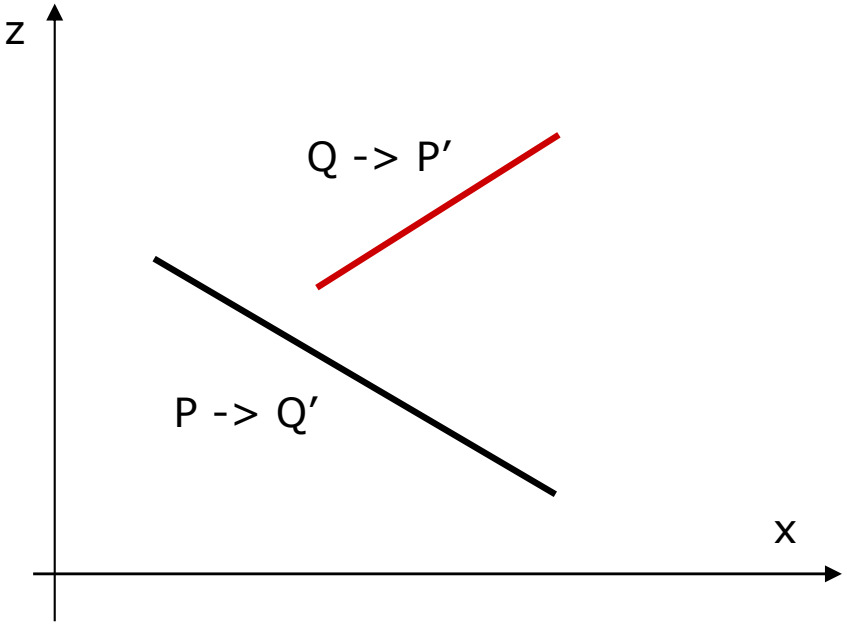


Viewing direction

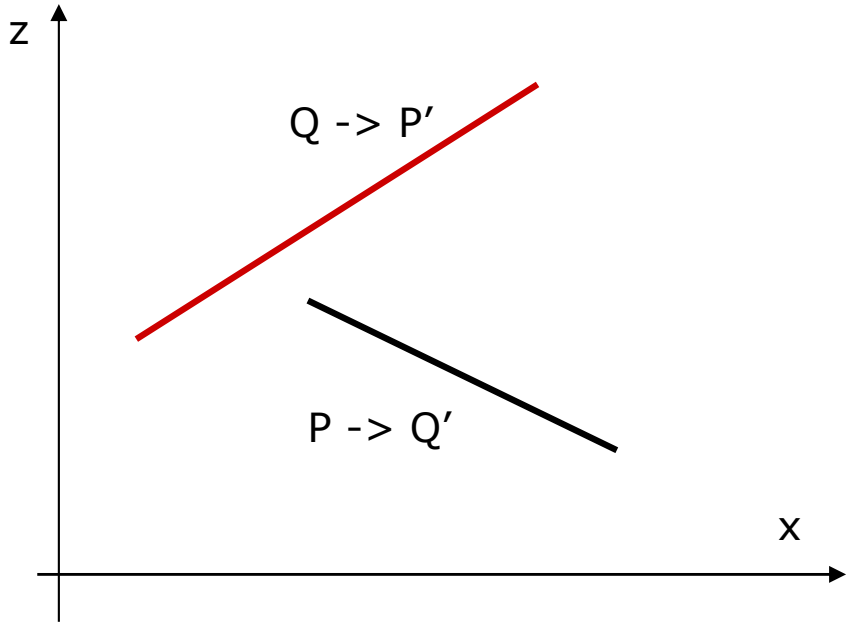
Test 5



Test 3* and 4* – swap P and Q



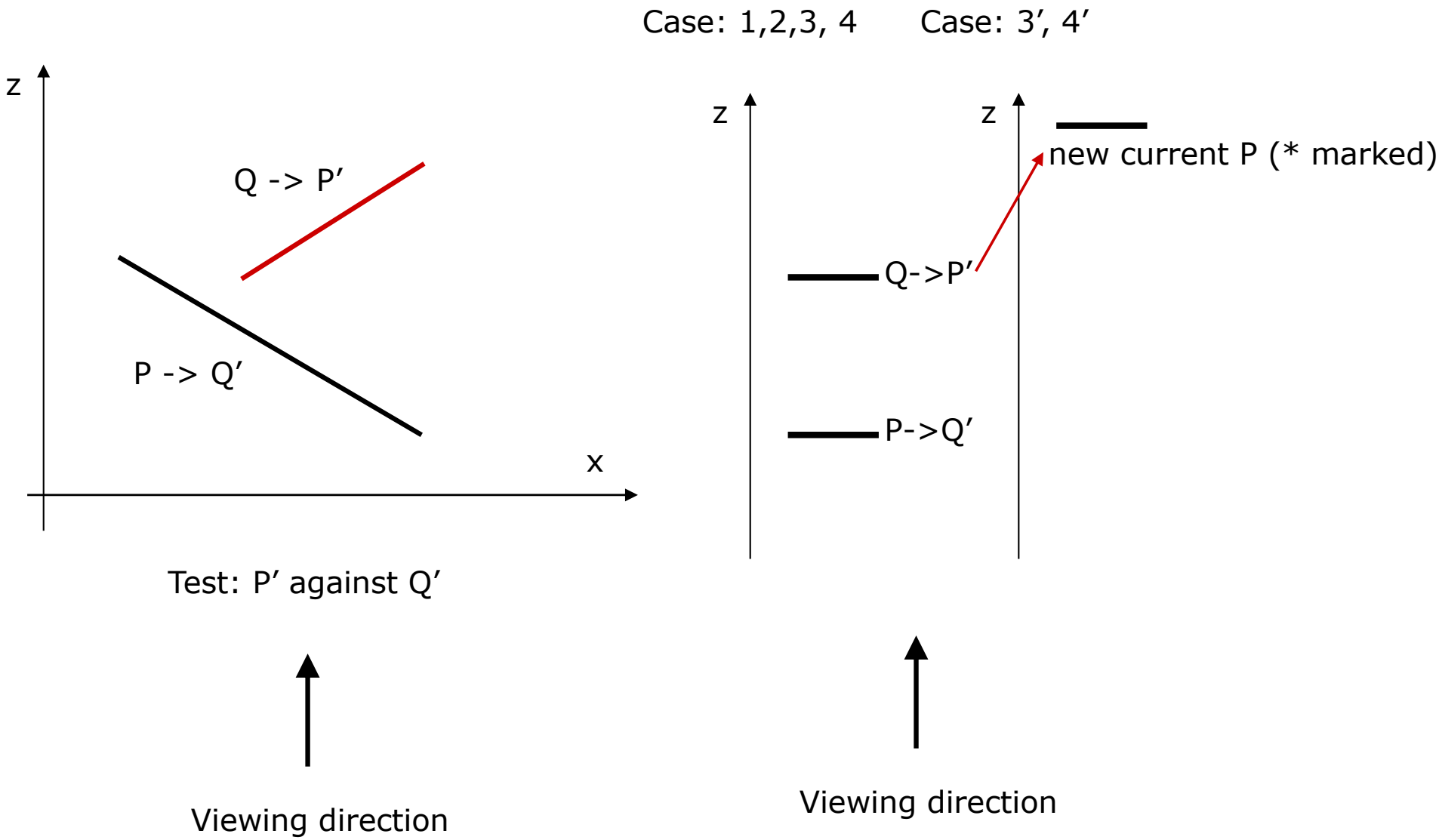
Test 3): test 3 for P and Q fails,
than test 3 for P’ and Q’, draw P’, Q’



Test 4): test 4 for P and Q fails,
than test 4 for P’ and Q’, draw P’, Q’

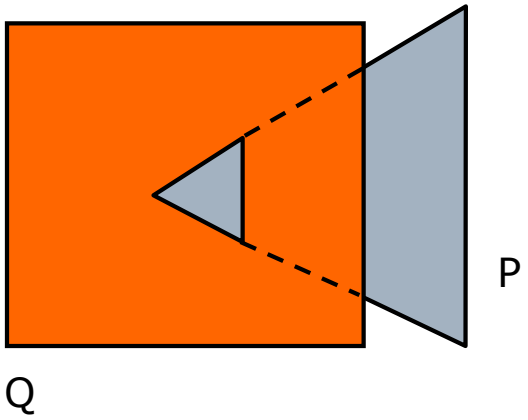
Viewing direction

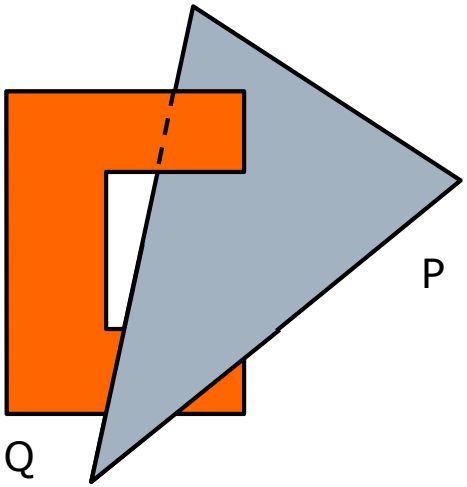
Test 3* and 4*

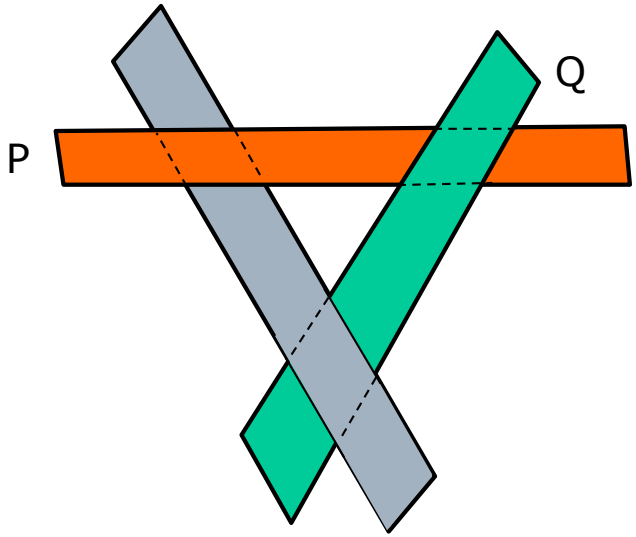

Case: 1,2,3, 4
Case: 3', 4'
z
Q -> P'
P -> Q'
x
Test: P' against Q'
Viewing direction
z
z
new current P (* marked)
Q->P'
P->Q'
Viewing direction

Cycle testing and splitting

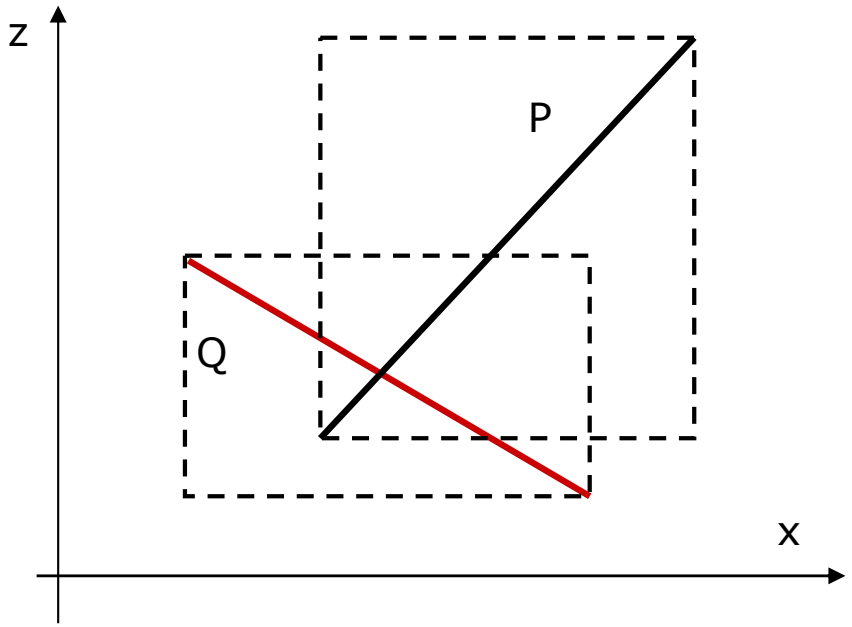
If not successful in box testing and swapping, then split P







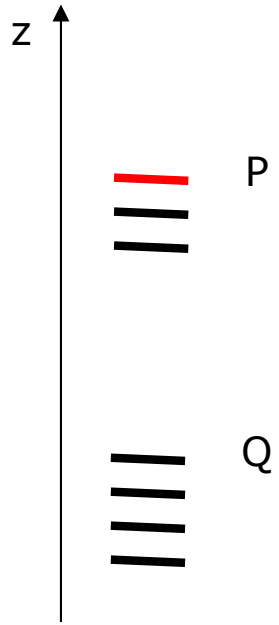
Split polygons



Test: P against Q

Viewing direction

Case: split polygons



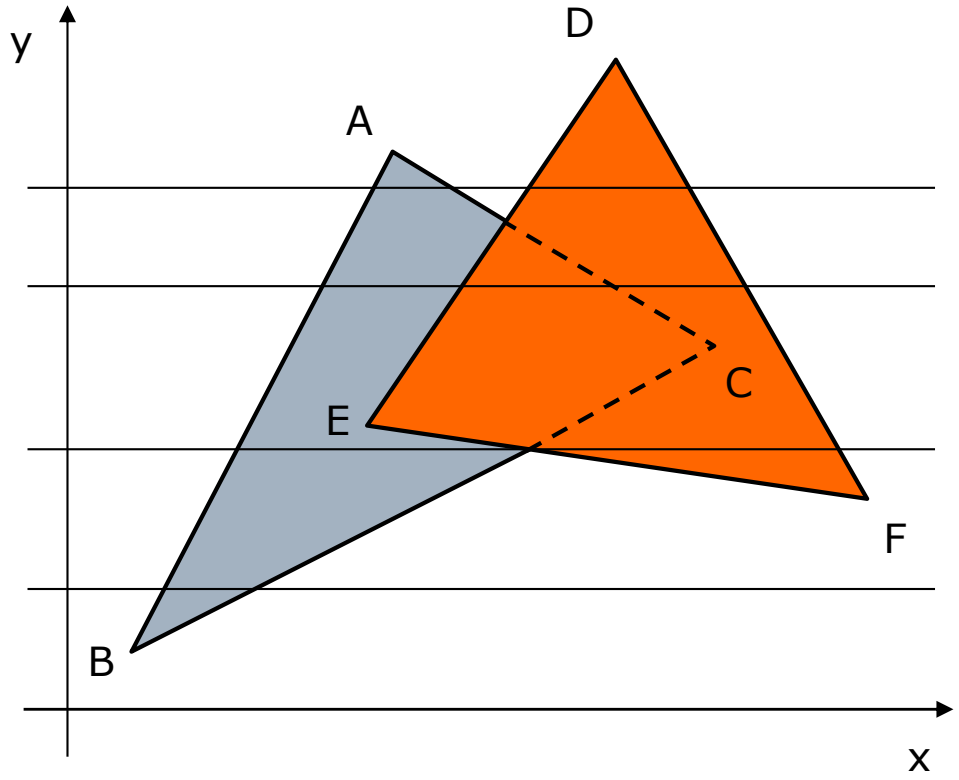
Viewing direction

6. Scan-Line Algorithms

- Wylie et al. (1967), W. Bouknight (1970), G. Watkins (1970)
- Published in 1967, by Wylie, C., Romney, G.W., Evans, D.C., Erdahl, A.C., "Halftone Perspective Drawings by Computer", FJCC 67.
- Published in 1970, by Bouknight, W.J., "A Procedure for Generation of Three-Dimensional Half-Toned Computer Graphics Presentations", CACM, vol 13(9).
- Published in 1970, by Watkins, G.S., "A Real Time Visible Surface Algorithm", PhD Thesis, Computer Science Dept., Univ. of Utah.
- Extension of the Ordered Edge List Algorithm (see course "Scan-Conversion(3)")
- Uses the scan line coherence and edge coherence
- Works with all polygons that define an object rather than a single polygon as in the Ordered Edge List Algorithm

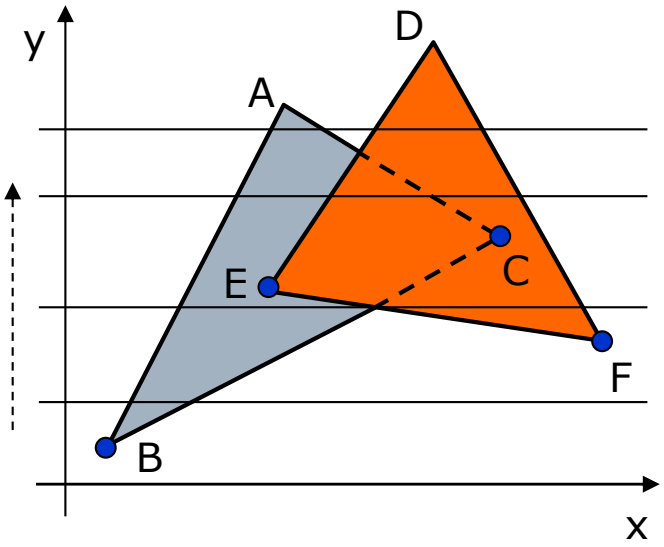
Algorithm steps

1. Create Edge Table (ET)
2. Create Polygon Table (PT)
3. Create Active-Edge Table (AET)



Create Edge Table (ET)

- All no horizontal edges
- Horizontal edges are ignored
- Edges sorted into buckets based on each edge's smaller y coordinate
- Within each bucket the edges are ordered by increasing x coordinate of their lower endpoint
- Each table entry contains:
  1. x coordinate of the end with the smaller y coordinate
  2. y coordinate of the edge's other end
  3. x increment Δx, is the step from one scan line to the next. Δx is the inverse slope of the edge (Δy = mΔx, Δy = 1, Δx = 1/m)
  4. Polygon identification number is the polygon to which the edge belongs

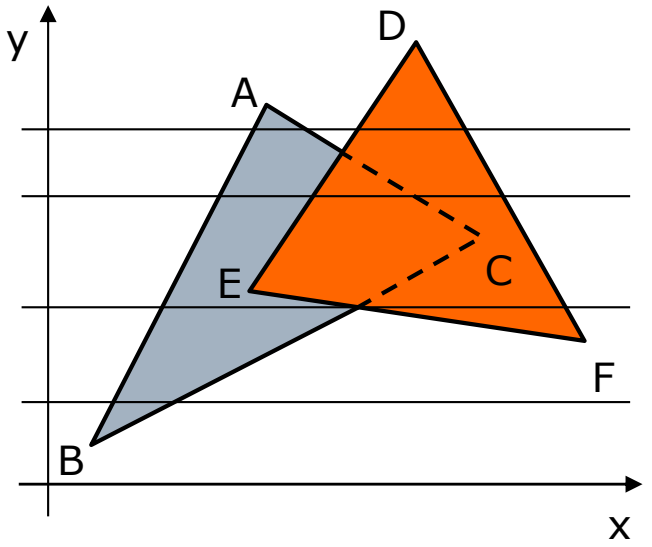


- ET:
  - AC
  - DE
  - EF, DF
  - AB, BC

x	ymax	Δx	ID	next edge

Create Polygon Table (PT)

- One entry for each polygon
- Each table entry contains the polygon ID
- Each polygon is described by at least the following information:
  1. Coefficients of the plane equation (i.e., A, B, C, D)
  2. Shading or color information
  3. An in-out Boolean flag, initialized to false



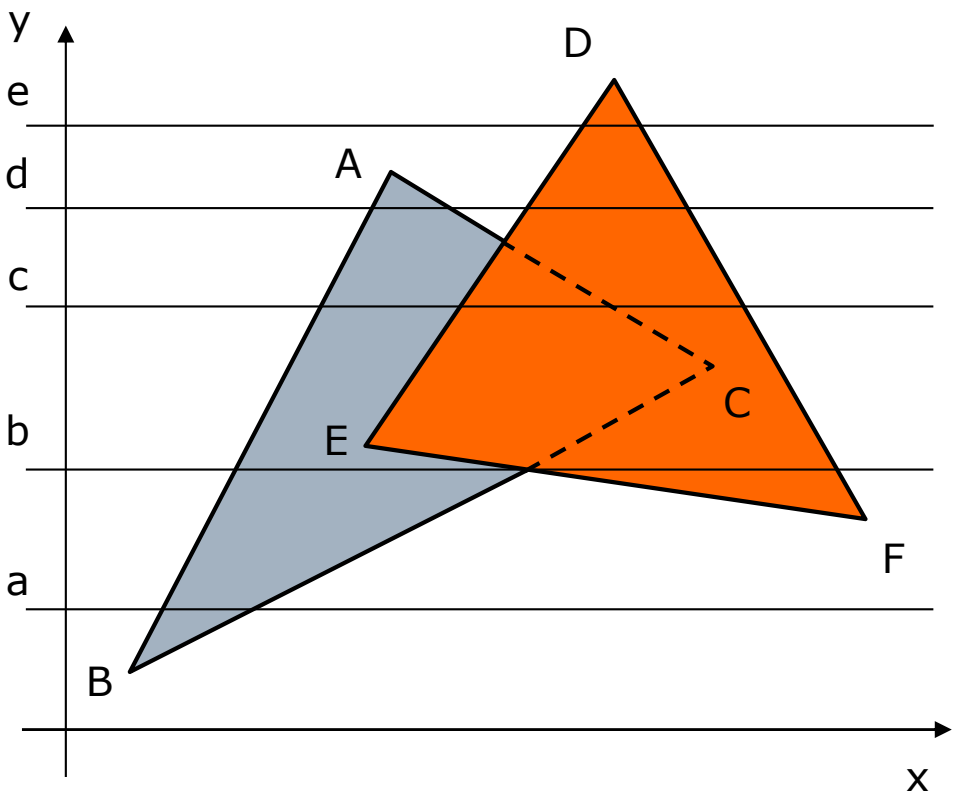
- PT:
  - ABC
  - DEF

ID	A,B,C,D	Shading info	in-out flag

Create Active-Edge Table (AET)

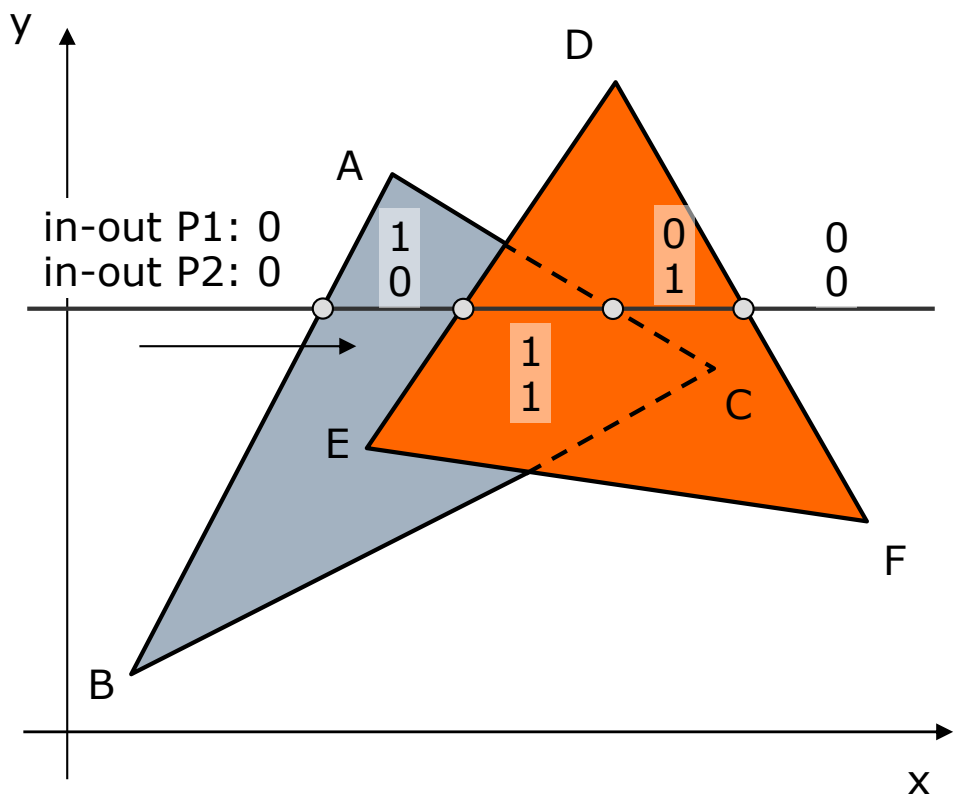
- Contains the edges intersected by the current scan line
- Edges are ordered by the x coordinate of the intersections between the active edges and the current scan line

- AET:
  - a: AB, BC
  - b: AB, EF, BC, DF
  - c: AB, DE, AC, DF
  - d: AB, AC, DE, DF
  - e: DE, DF



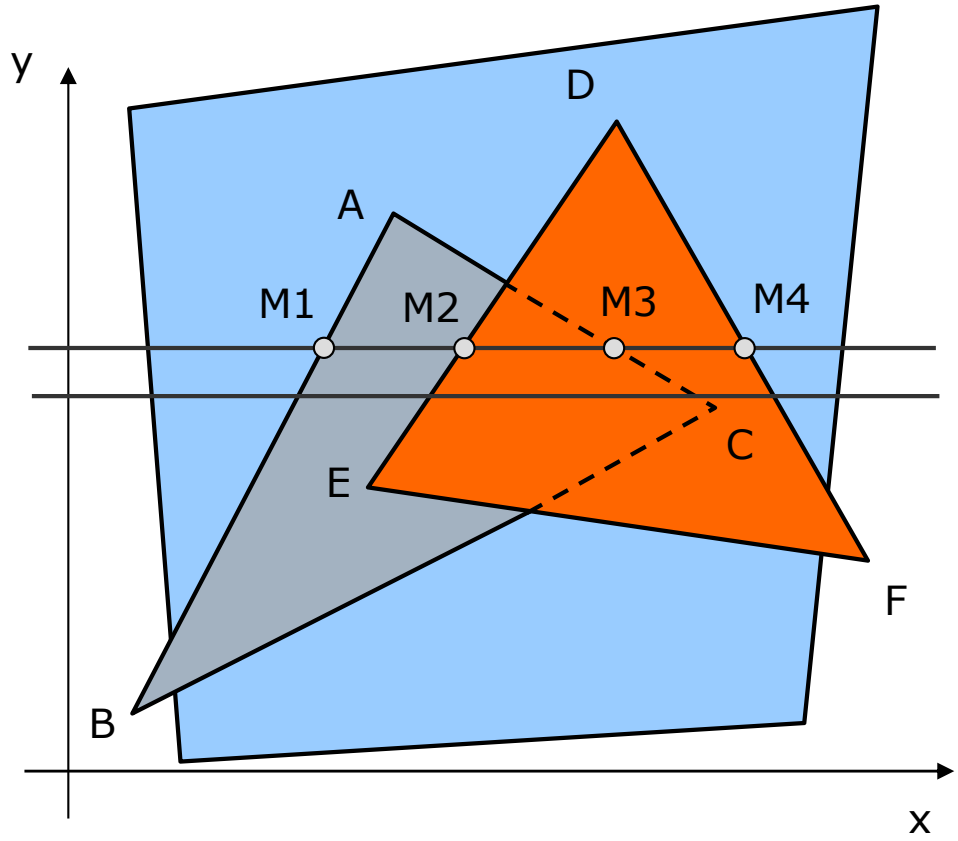
Scan line parsing

- Pixel to pixel, left to right
- Update in-out flags
- Compute the depth order at the intersections
  - Visible polygon
  - Graphics attributes (e.g., Pixel color, texture, filling pattern, etc.)



Scan line algorithm optimization

- Scan line coherence:
  - If polygons do not intersect each other (in object space)
  - Then, on M3 the depth calculation can be avoided
  - Order in M2 remains the same until in M4
- Depth coherence
  - If polygons do not penetrate each other
  - If the same edges are in the AET on one scan line as are on the preceding scan line and in the same order
  - Then, no changes in depth relationships on any part of the scan line
  - No new depth computations are needed



Scan line algorithm extension

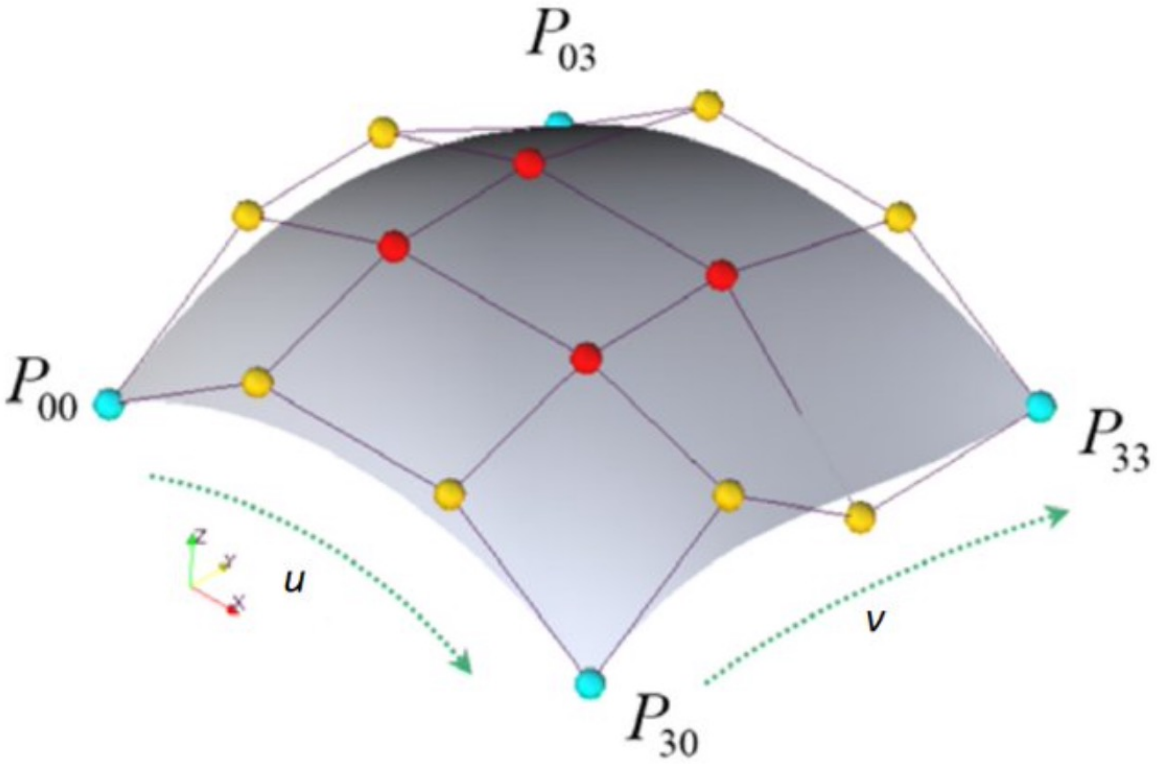
- Extend the Scan Line Algorithm to general surfaces (curved surfaces)
- ET replaced by ST (Surface Table)
- AET replaced by AST (Active Surface Table) sorted by the (x,y) extents of the surfaces
- When a surface is moved from ST to AST it is decomposed into a set of polygons
- When a surface leaves the AST, all associated polygons are discarded

```
add surfaces to ST;
initialize AST;
for each scan line {
    update AST;
    for each pixel on scan line {
        determine surfaces in AST that project to pixel;
        find closest such surface;
        determine the graphics attribute of the closest surface;
    }
}
```

Scan line algorithm extension

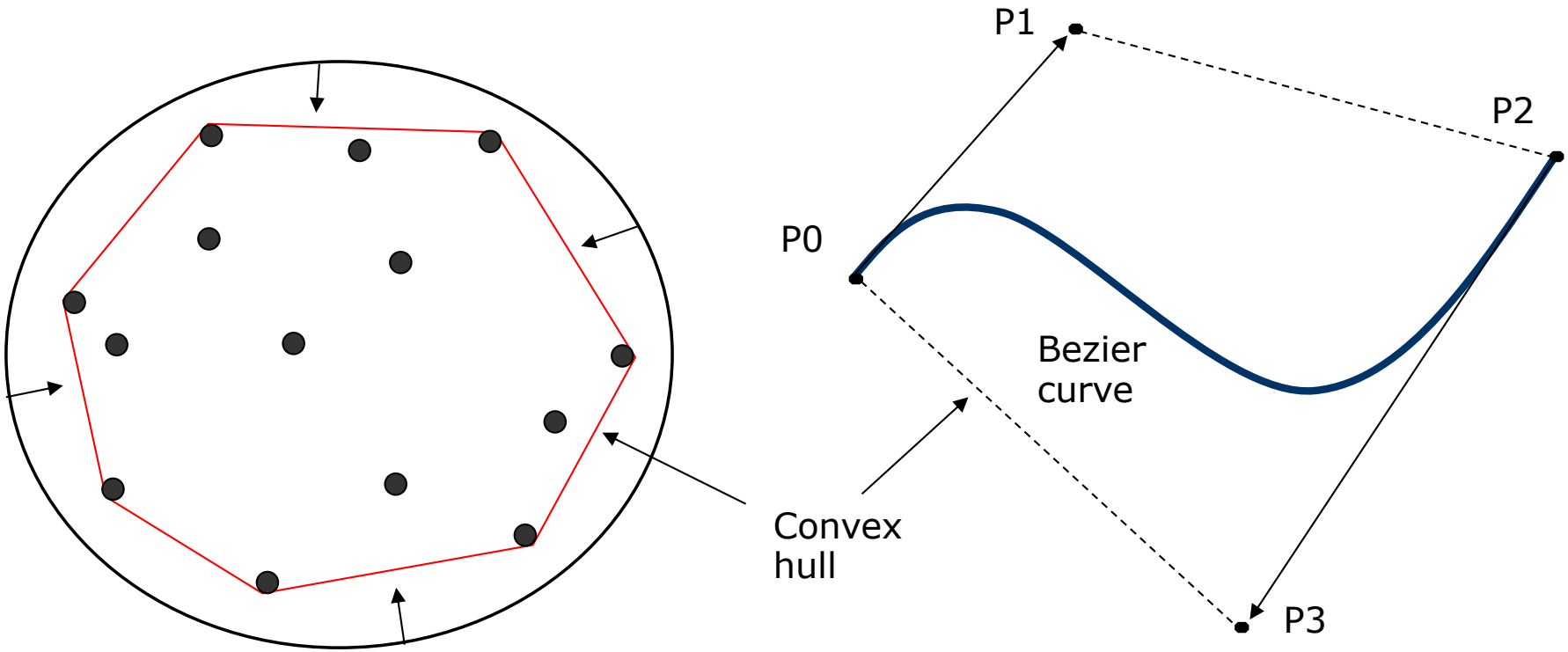
- Scan line z-buffer algorithm
  - Meyer, A.J. (1975), published by "An Efficient Visible Surface Program", Report to the National Science Foundation, Ohio State University.
  - uses a z-buffer to resolve the visible surface problem
  - A single scan line frame buffer and z-buffer is used for each scan line
- Scan line algorithms for curved surfaces
  - Lane-Carpenter Algorithm
  - Published in 1980, by Lane, J., Carpenter, L., Whitted, T, and Blinn, J. "Scan Line Methods for Displaying Parametrically Defined Surfaces", Communications of the ACM.
  - Uses Patch Table (PT) and Active Patch Table (APT)
  - Does the subdivision only as required when the scan line begins to intersect a patch, rather than in a preprocessing step
  - The patch position is analyzed by the patch's control points that define its convex hull

Bézier patch

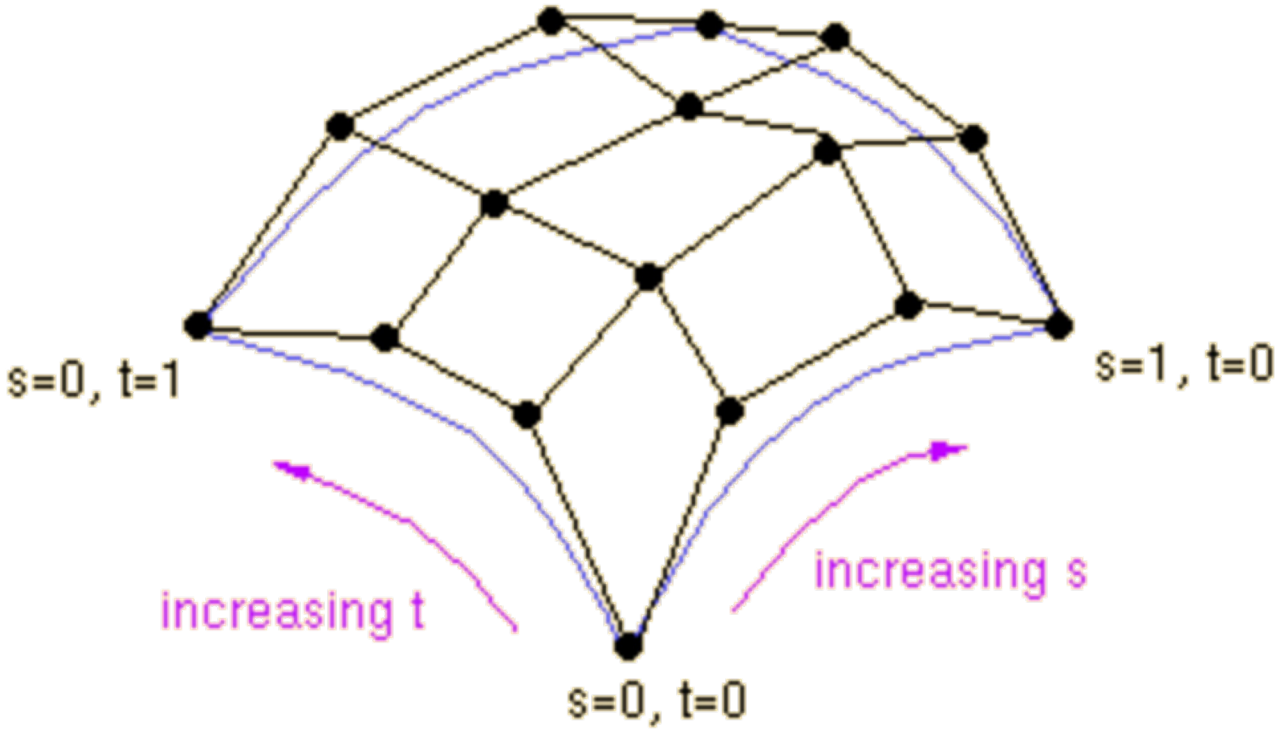


Convex hull property

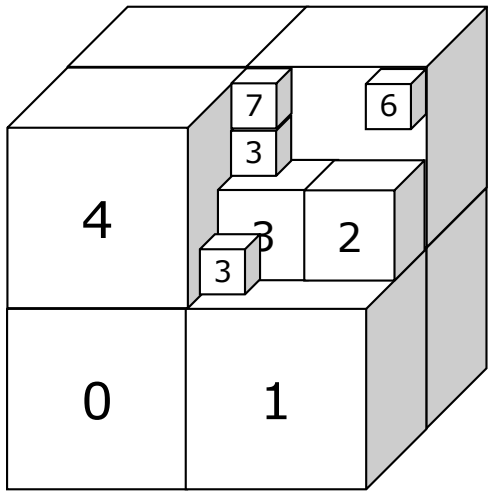
- Convex hull or convex envelope for a set of points *X* in a real vector space V is the minimal convex set containing *X*.
- A Bézier curve is circumscribed by the quadrilater defined by its four control points.
- The Bézier curve as well as the Bézier surface could be approximated by the adjacent convex hull polygons and rendered as a polygonal surface.

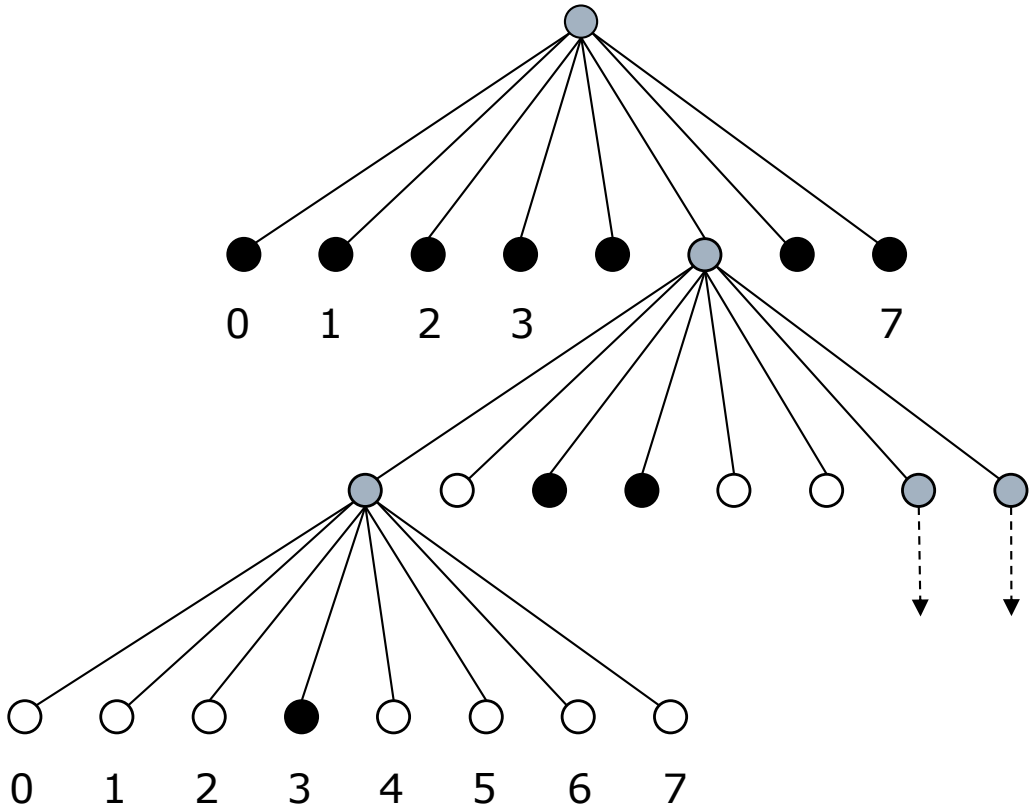


Convex hull of Bézier surface

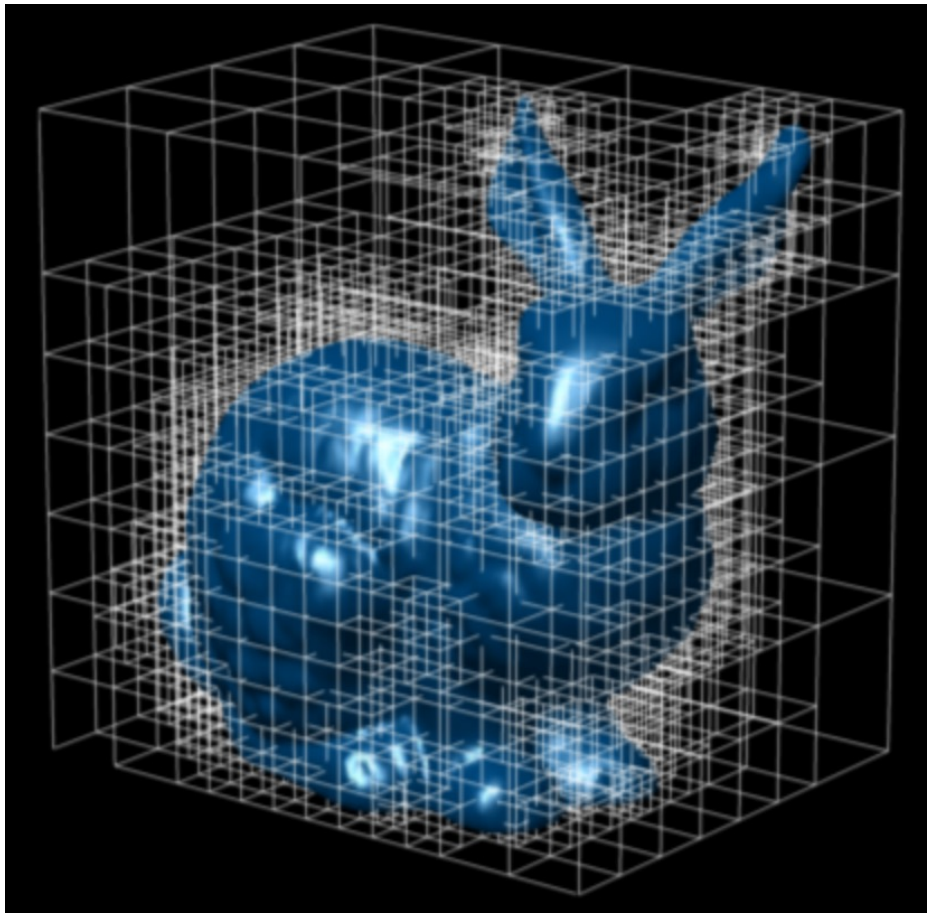


7. Octree Based Algorithms





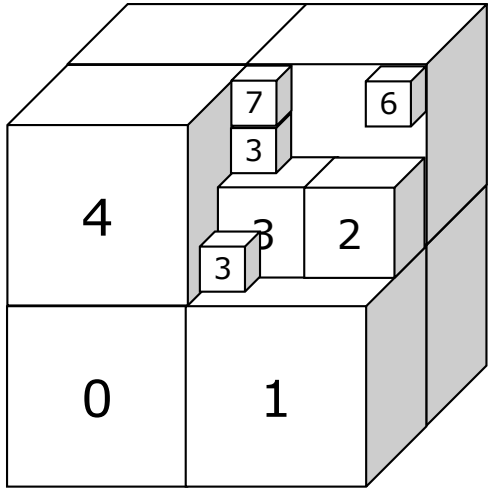
Octree model

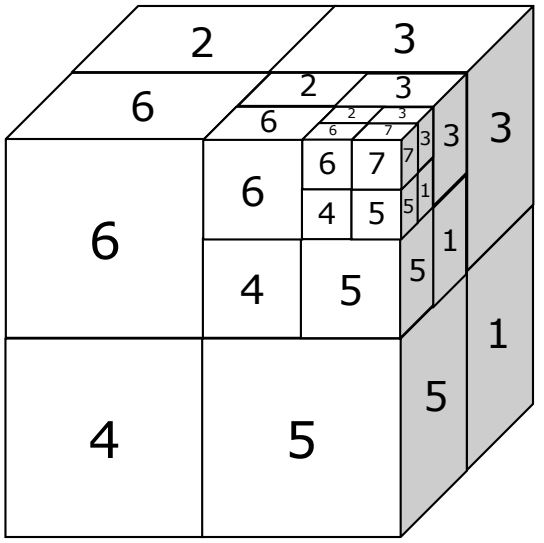


Sven (2021). octree - partitioning 3D points into spatial subvolumes (https://www.mathworks.com/matlabcentral/fileexchange/40732-octree-partitioning-3d-points-into-spatial-subvolumes), MATLAB Central File Exchange. Retrieved October 31, 2021.

Octree encoded objects

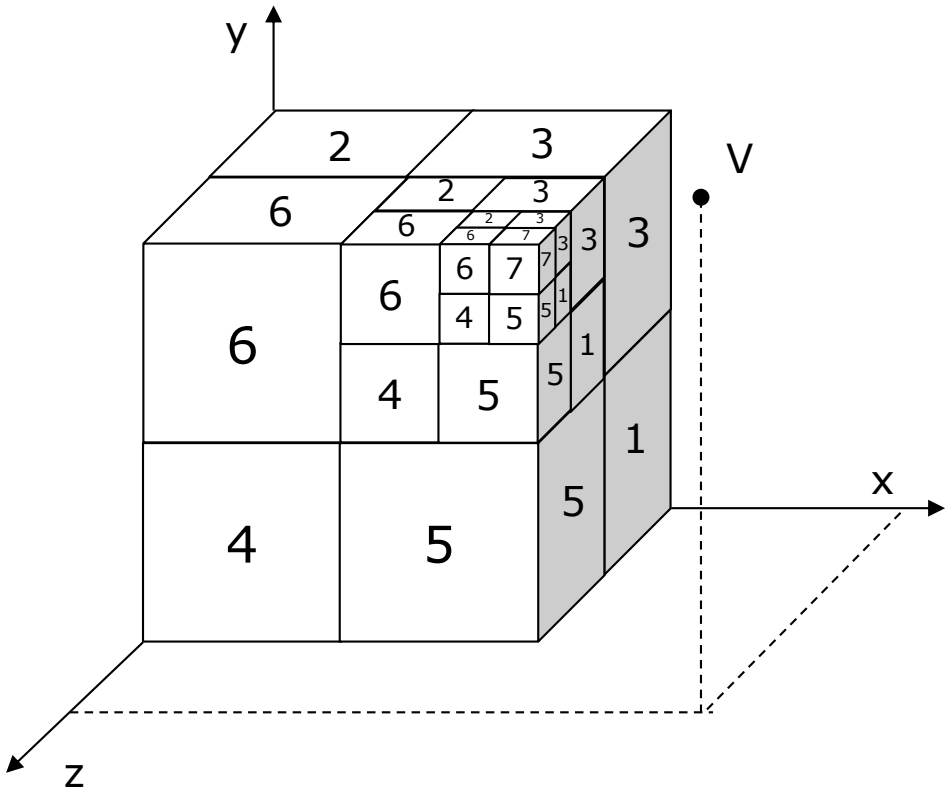
- Octree regular structure gives nonintersecting cubes
- Octree is partially pre-sorted
- Octree enumeration could improve the operations (i.e. visibility computation):
  - Slice based enumeration
    - Iterate through the octants in slice perpendicular to one of the axes
    - Orthographic projection
    - E.g. variate z, then y, and then x
  - Back to front enumeration
    - The nodes are listed in order in which any node is guaranteed not to obscure any node listed after it
    - Orthographic projection
    - Display order: (1) the farthest octant, (2) three neighbors of the closest octant in any order, (3) closest octant





Octree enumeration for visibility

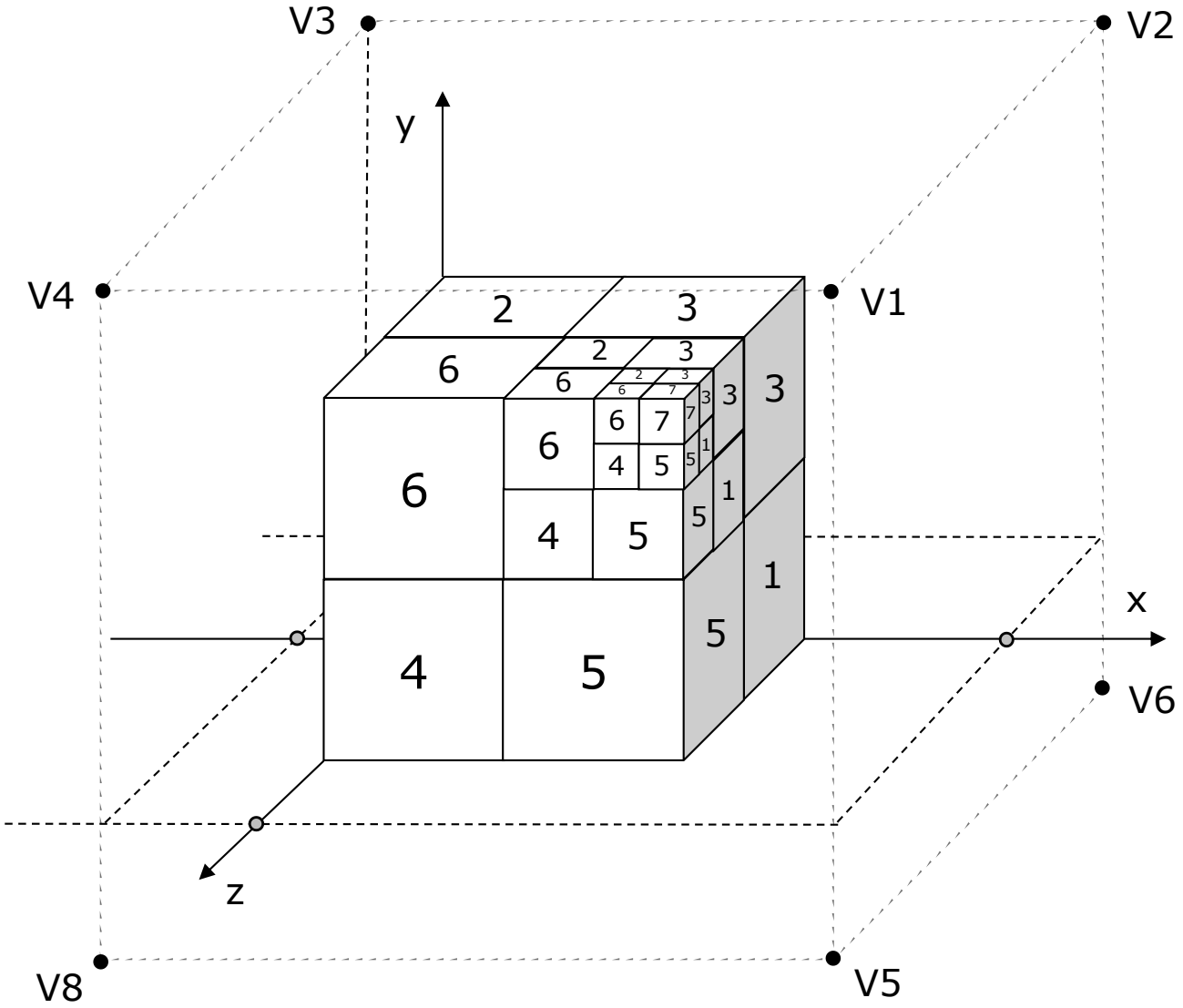
- Eight back to front orders given by the viewer's relative position
- Positive x coordinate means the right (R) face is visible, and negative x means the left (L) face is visible
- Similarly: y+/- gives up(U)/down(D) face, z+/- gives front(F)/back(B) face



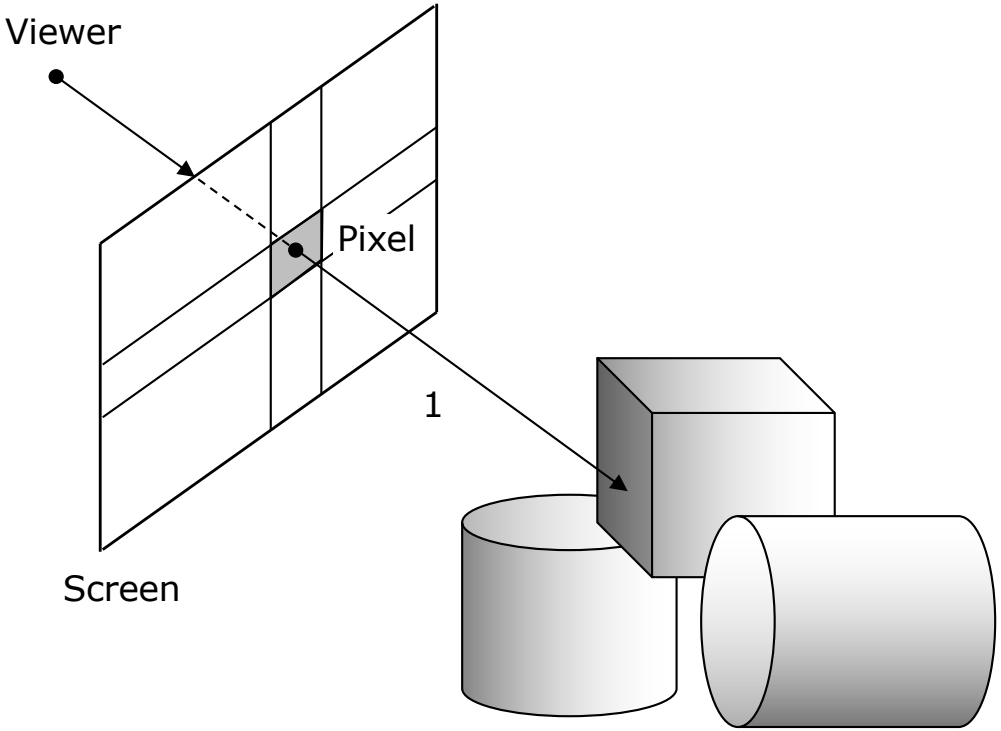
Back to front enumeration and visible faces

z	y	x	Back to front order	Visible faces
-	-	-	7,6,5,3,4,2,1,0	B,D,L
-	-	+	6,7,4,2,5,3,0,1	B,D,R
-	+	-	5,4,7,1,6,0,3,2	B,U,L
-	+	+	4,5,6,0,7,1,2,3	B,U,R
+	-	-	3,2,1,7,0,6,5,4	F,D,L
+	-	+	2,3,0,6,1,7,4,5	F,D,R
+	+	-	1,0,3,5,2,4,7,6	F,U,L
+	+	+	0,1,2,4,3,5,6,7	F,U,R

Octree enumeration for visibility



8. Ray Casting Algorithms

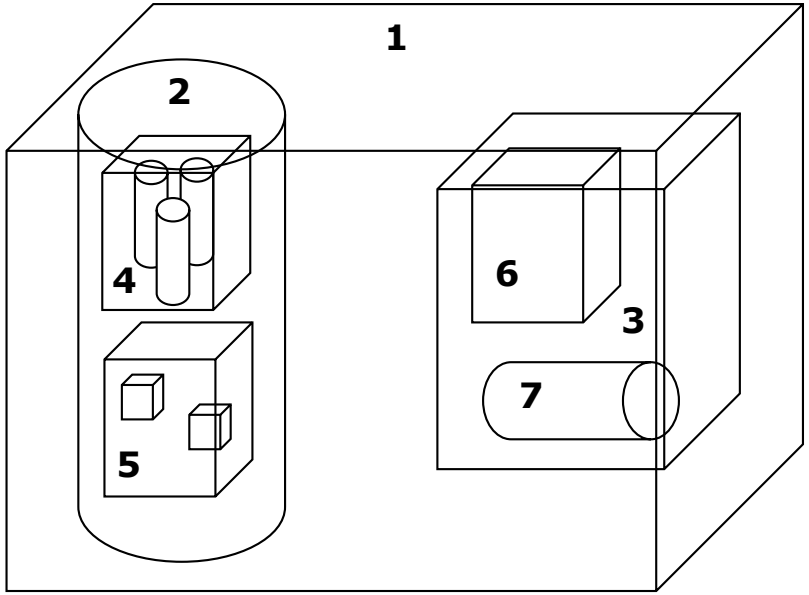


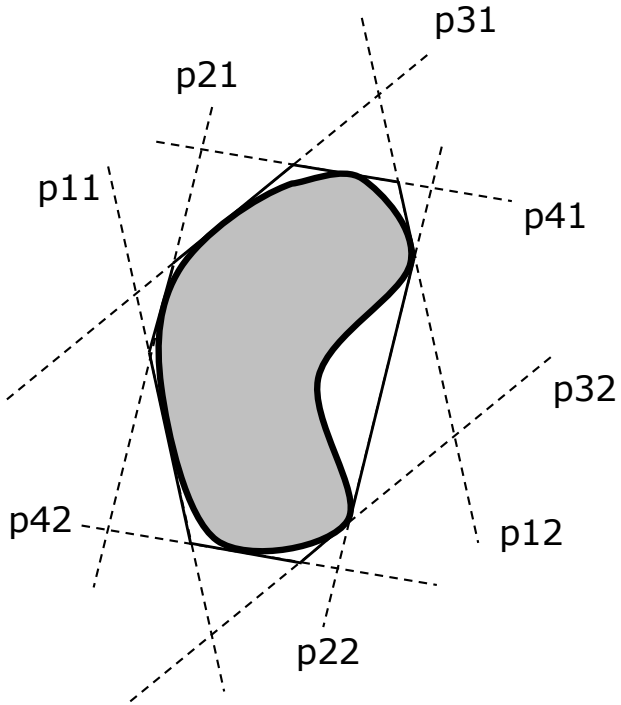
- Ray tracing, ray casting
- Developed by Appel (1968), Goldstein and Nagel (1968), Whitted (1980), and Kay (1979)
  - Appel: uses a sparse grid of rays to compute the shadows
  - Goldstein and Nagel: simulate the trajectories of ballistic projectiles and nuclear particles, and later apply it in graphics
  - Whitted and Kay: extended ray tracing to handle specular reflection and refraction

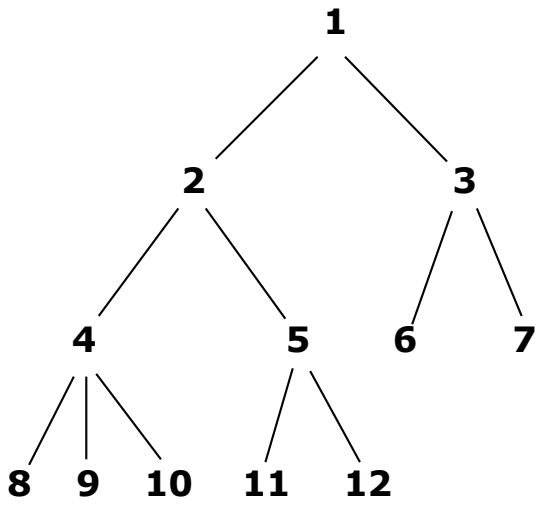
Ray casting issues

- Computing intersections
  - Ray (parametric line) and a polygon
  - Line and cylinder, cube, sphere, etc.
  - Line and quadrics (e.g., ellipsoid, paraboloid, etc.)
- Efficiency for visible surface ray tracing
  - Optimize the intersection computation
  - Avoid intersection computation
  - Hierarchical bounding volumes
  - Efficient hierarchy traversal
  - Automatically hierarchy generation
  - Spatial partitioning
- Computing Boolean set operations
  - Compute the visibility for 3D union, difference, and intersection of two solids (e.g., on CSG hierarchy)
- Antialiasing ray tracing
  - Point sampling on a regular grid produces aliasing images
  - Adaptive supersampling (additional samples by more rays), e.g., in corners of each pixel.

Ray casting issues

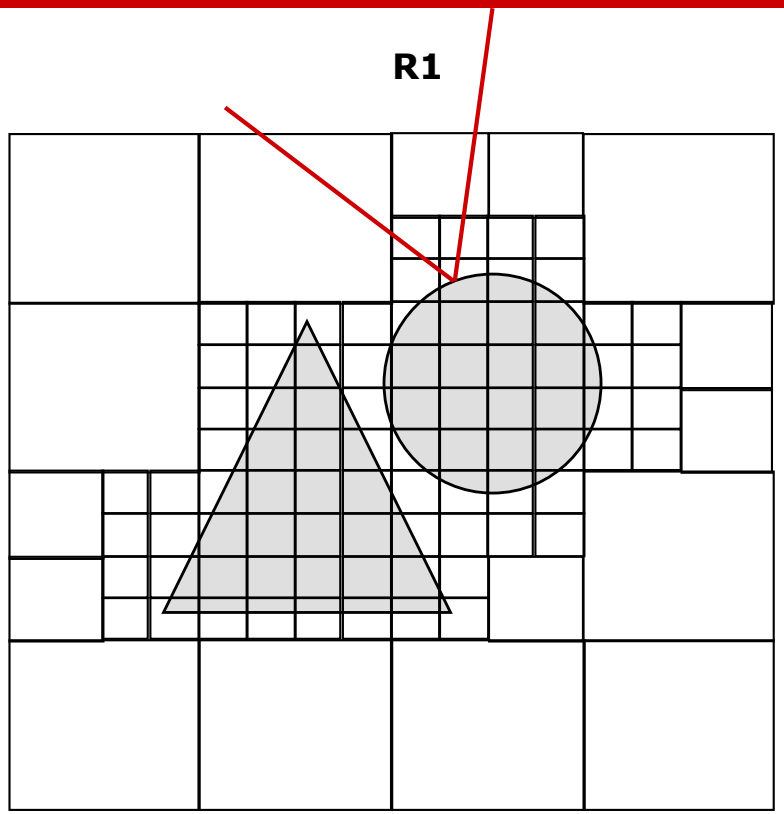


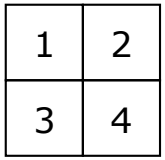


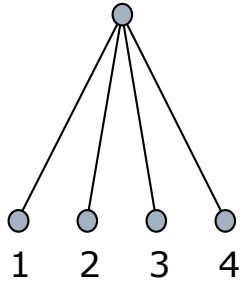


- Hierarchical bounding boxes
- Efficient extents

Ray casting issues

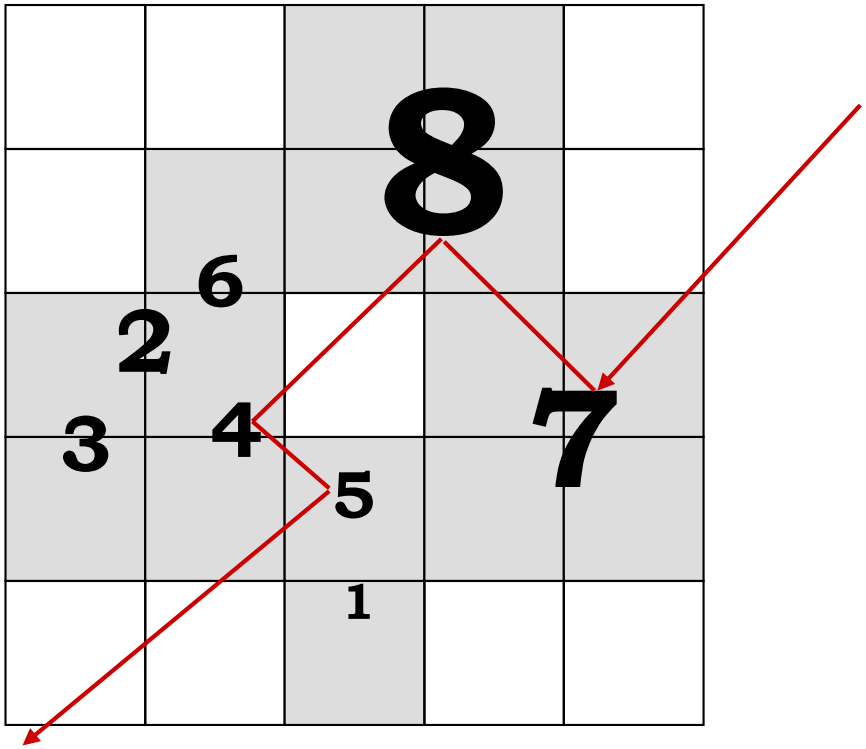






- Quadtree, octree
- Automatically hierarchy generation
- Spatial partitioning
- Efficient hierarchy traversal
- Optimize the intersection computation

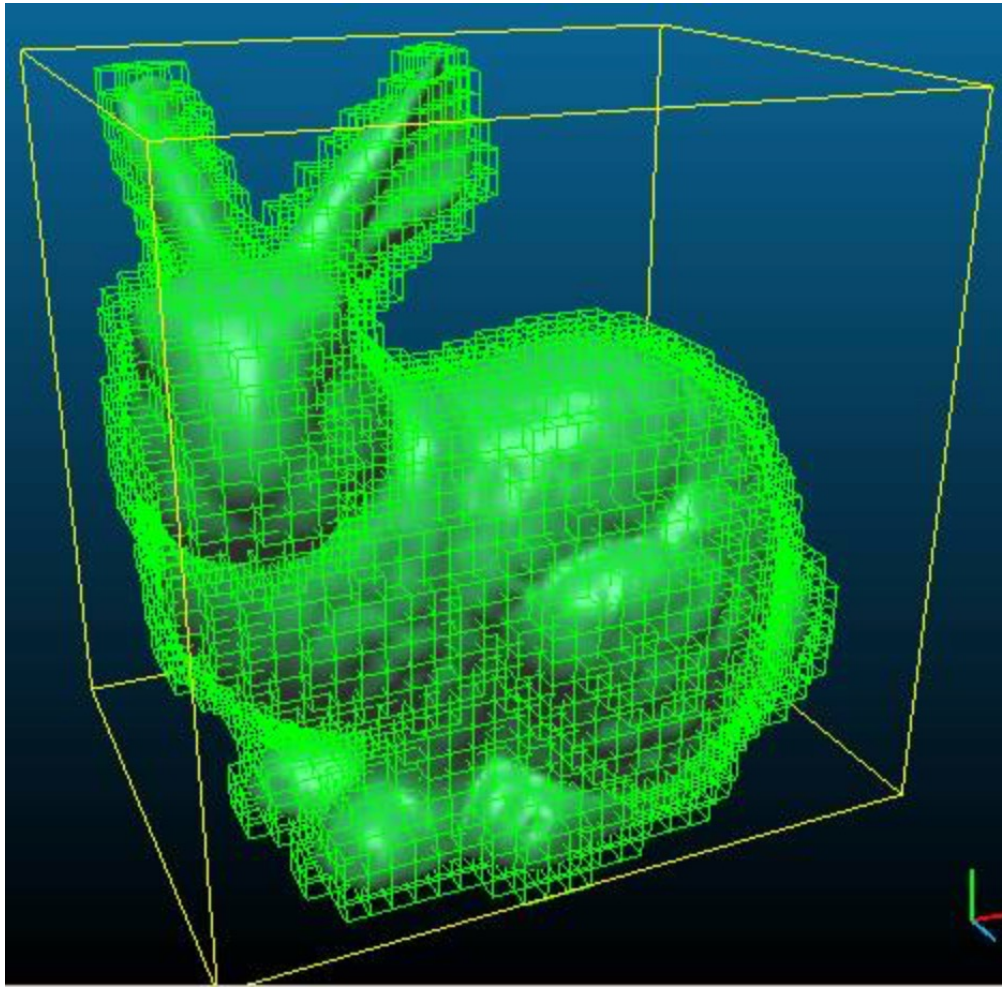
Ray casting issues



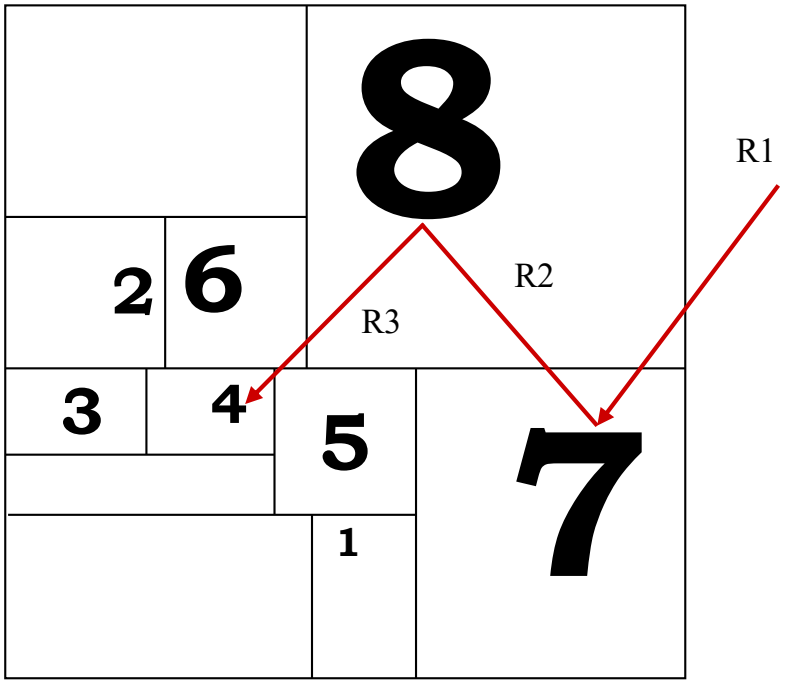
		8	8	
	6	8	8	
23	246		7	7
3	4	15	7	7
		1	8	

- Automatically hierarchy generation
- Spatial partitioning
- Efficient hierarchy traversal
- Optimize the intersection computation

Equal spatial partitioning



Ray casting issues

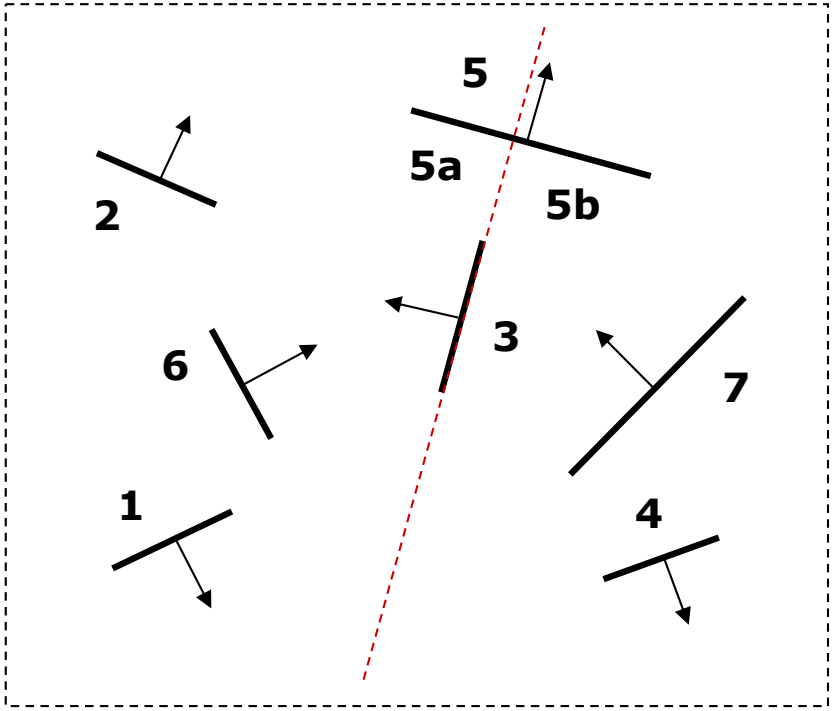


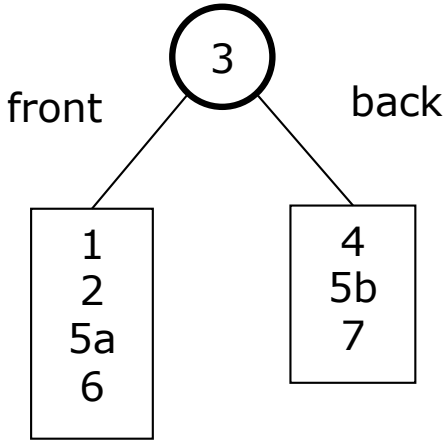
- Automatically hierarchy generation
- Spatial partitioning
- Efficient hierarchy traversal
- Optimize the intersection computation

9. BSP Trees

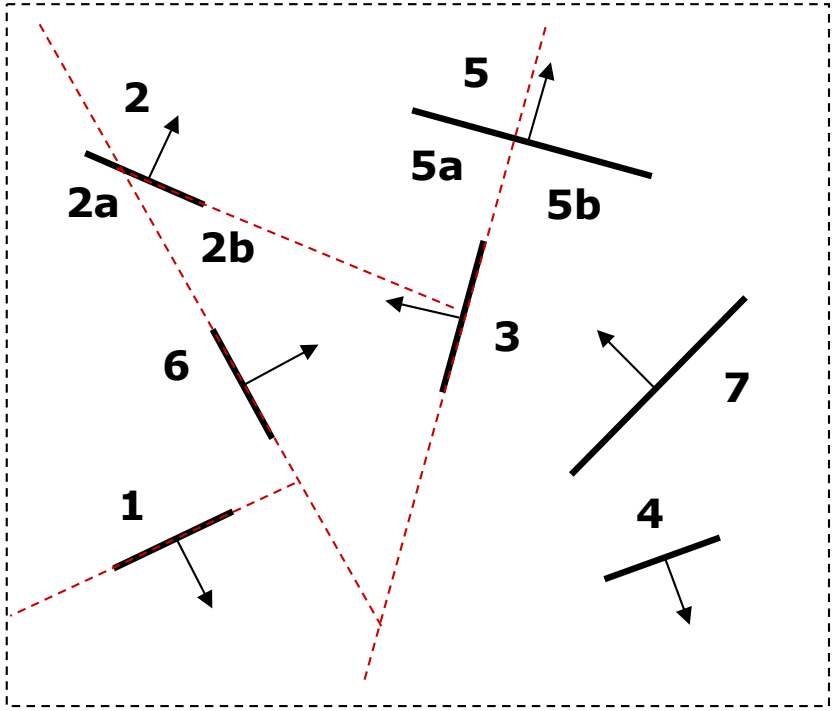
- Binary Space Partitioning Trees (BSP Trees)
- Developed by Fuchs, Kedem and Naylor in 1980
  - Published in 1980, by Fuchs, H., Kedem, Z.M., Naylor, B.F., "ON Visible Surface Generation by A Priori Tree Structures", SIGGRAPH 80.
- The basic idea has been published in 1969 by Schumacker, who considered the space as a set of clusters separated by planes (faces):
  - Clusters that are on the same side of the plane as the eyepoint can obscure, but cannot be obscured by clusters on the other side
- Suited for applications in which the viewpoint changes, but the objects do not
- Steps:
  1. Build the BSP tree
     1. Compute polygon planes
     2. Determine the normal vector for each polygonal face
     3. Chose the root node
     4. Add nodes by analyzing the relative position of other faces relative to the current parent node
  2. Visualize the polygons in order infered from the BSP tree and the viewer's position

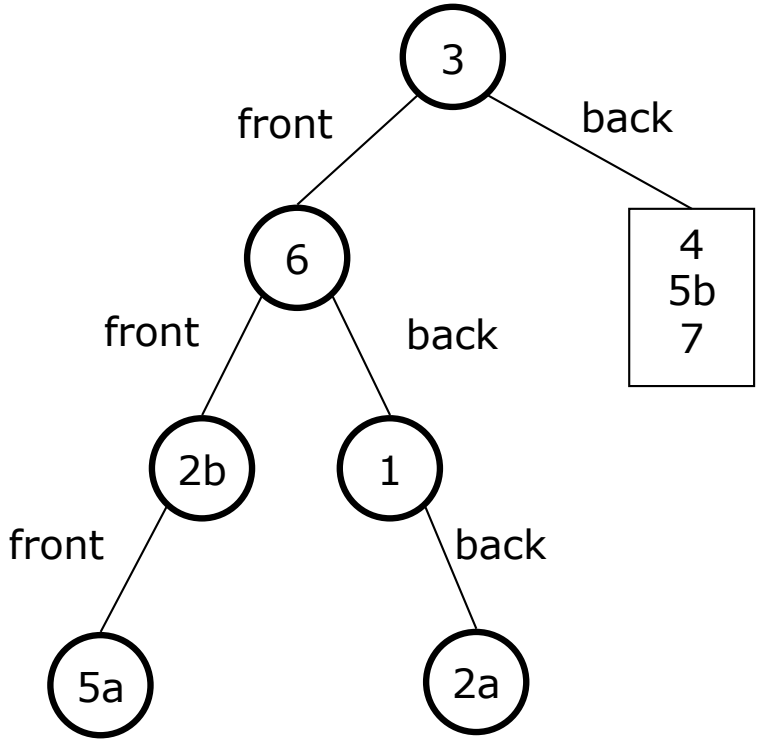
BSP Trees - Example



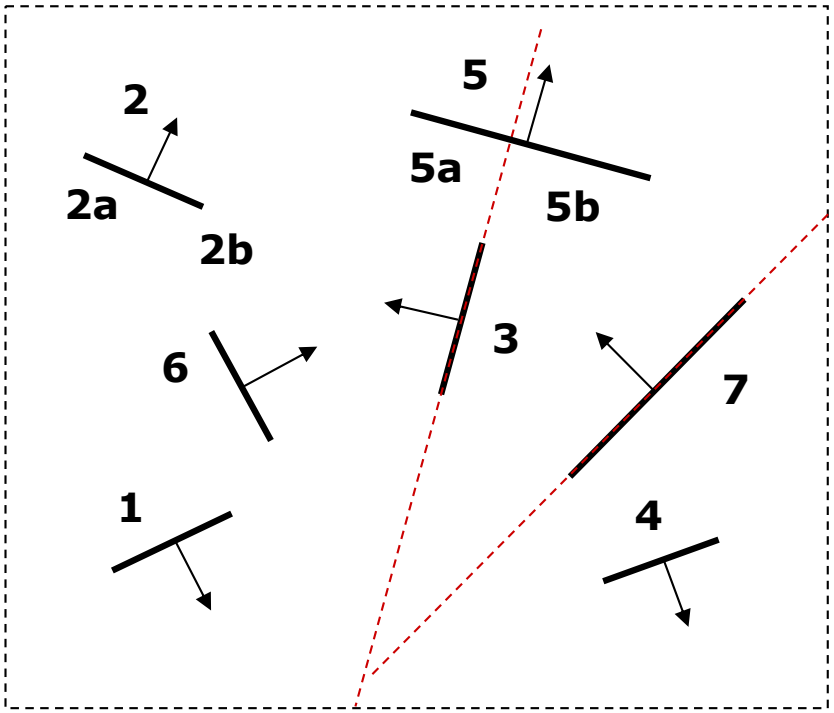


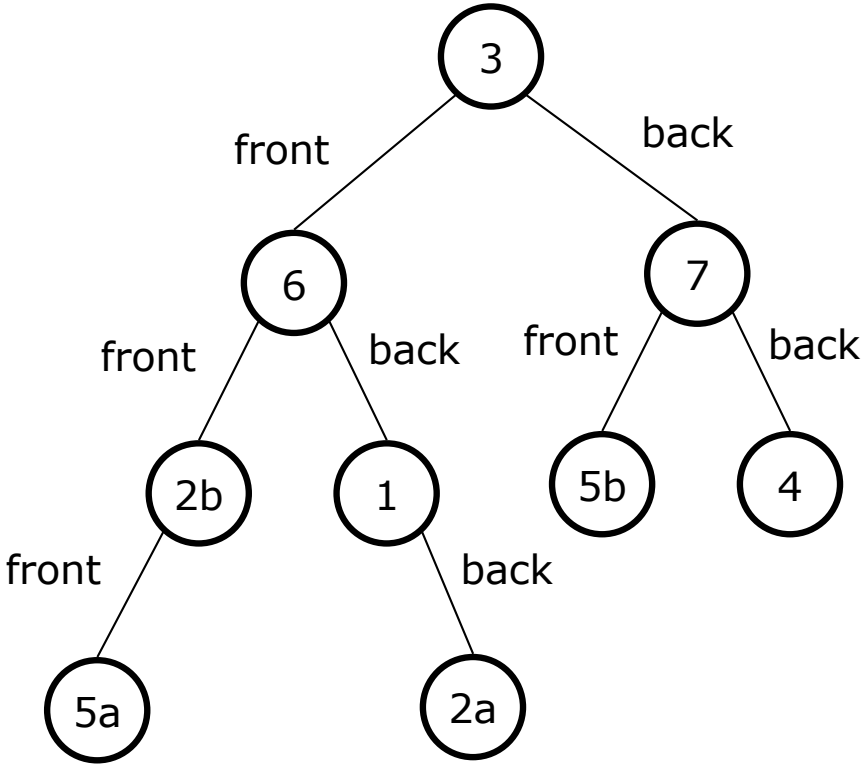
BSP Trees - Example



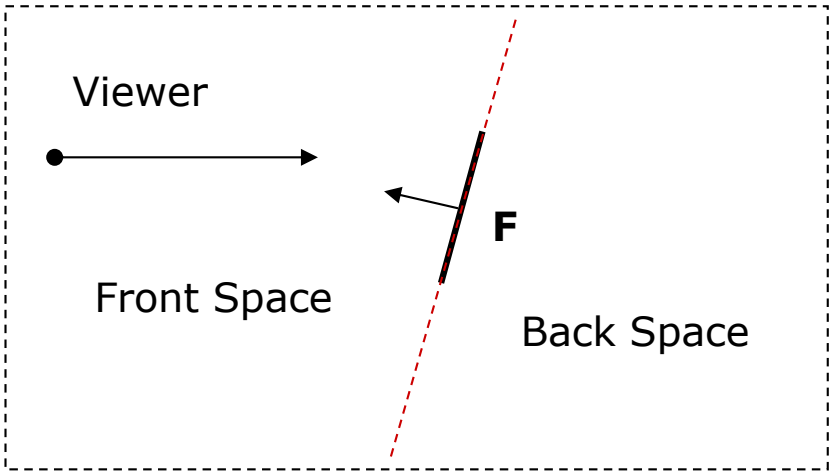


BSP Trees - Example



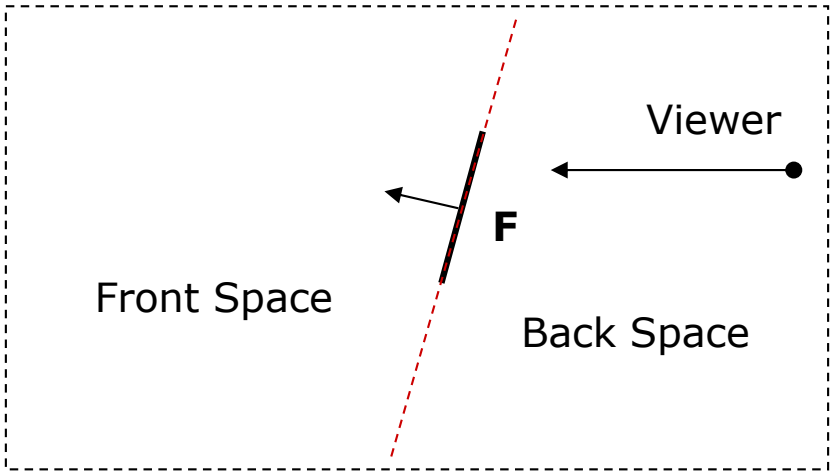


BSP Trees - Display



Display order:

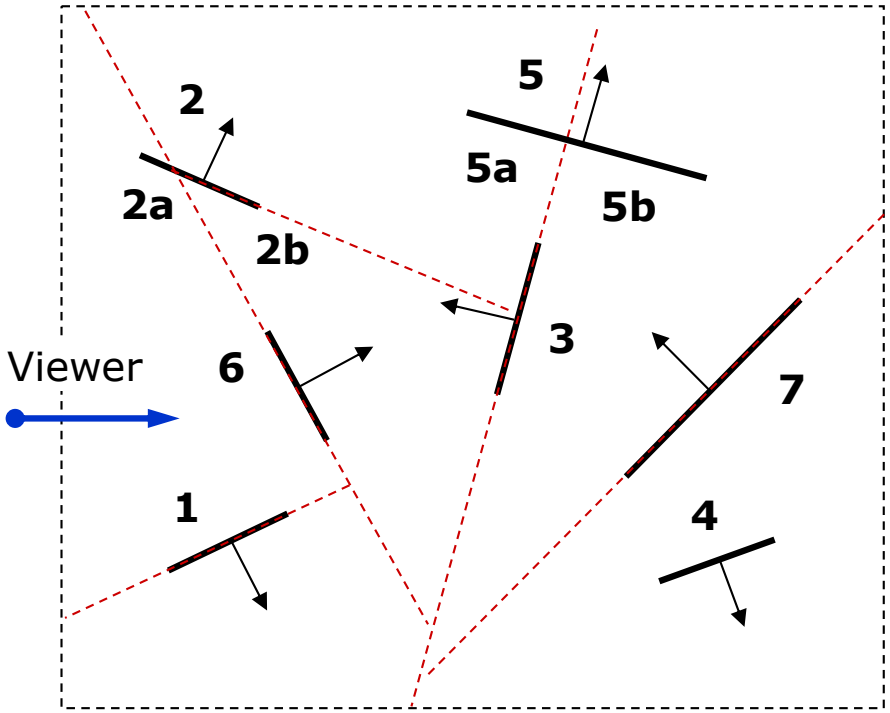
1. Back space polygons
2. Root face
3. Front space polygons

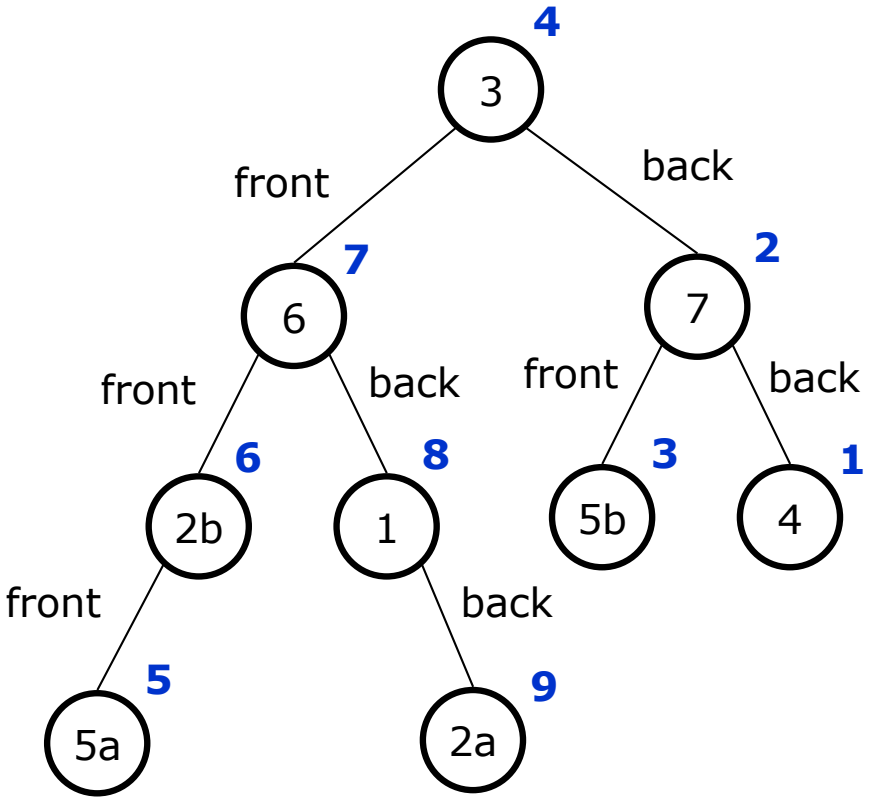


Display order:

1. Front space polygons
2. Root face
3. Back space polygons

BSP Trees - Display





Display order: 4, 7, 5b, 3, 5a, 2b, 6, 1, 2a

Questions and problems

1. Let us consider the user's position P on axe z at infinity. Determine if the relative positions of two 3D polygons A and B are in one of the following cases:
   a. Point P and polygon A are in the same half space of polygon B
   b. Point P and polygon A are in opposite half space of polygon B
   c. Polygon A intersects the plane of polygon B
   d. Polygons A and B are within the same plane
2. Let us consider a cube C in the domain of positive z having one main diagonal parallel with axe z. The projection plane is xy, and the user is located on axe z at infinity.
   a. Exemplify and explain the Depth Sorting algorithm for the cube C
   b. Exemplify and explain the Scan Line algorithm for the cube C

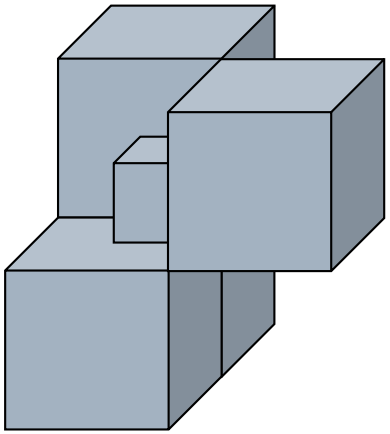
Questions and problems

3. Why, in the Scan Line algorithm, the edges are sorted into buckets based on each edge's smaller y coordinate? In what case the bucket could be defined for the bigger y coordinate?
4. Why, in the Scan Line algorithm, within each bucket the edges are ordered by increasing x coordinate of their lower endpoint?
5. Explain how the concept of coherence is used in the Scan Line algorithm. What is the property that is propagated from one step to the next one?
6. Exemplify and explain how works the Scan Line algorithm in the case of more than three polygons.
7. What are the main issues of the Ray Casting algorithm working on the scene of: a) static objects; b) moving objects; c) deformable objects.

Questions and problems

8. Generate the octree for the complex object in the figure 1, composed of cubes.
9. Exemplify and explain the method “Octree enumeration for visibility” for the complex object in the figure 1, composed of cubes.

Figure 1.



Questions and problems

10. Generate the octree for the object in the figure 2, composed of one triangle.
11. Exemplify and explain the method “Ray Casting” for the visualization of the object in the figure 2 (composed of just one triangle).
12. Exemplify and explain the method “Ray Casting based on the octree model” for the visualization of the object in the figure 2 (composed of just one triangle).

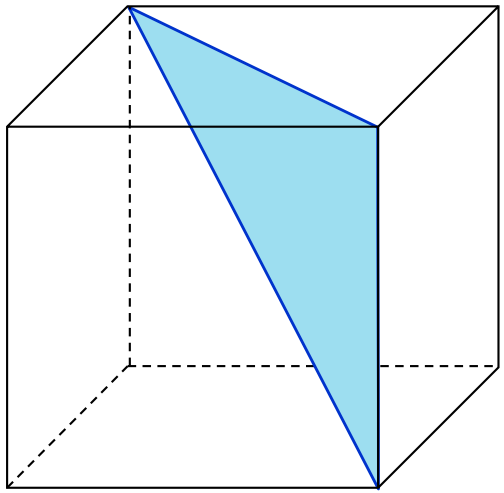


Figure 2.

Questions and problems

13. Generate the octree for the object in the figure 3, composed of two triangles.
14. Exemplify and explain the method "Ray Casting based on the octree model" for the visualization of the object in the figure 3 (composed of two triangles).

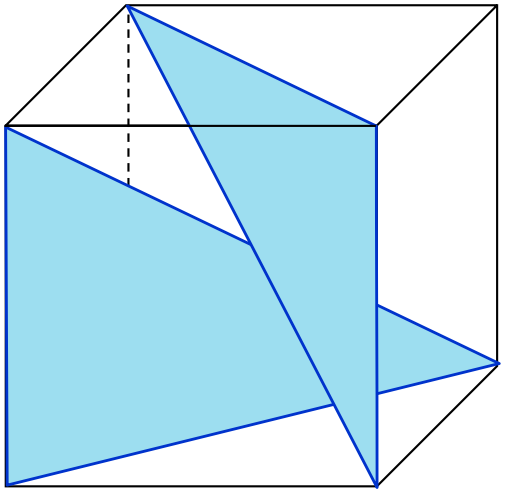


Figure 3.

Questions and problems

15. Exemplify and explain the method "Ray Casting based on the octree model" for the visualization of the object in the figure 4 (composed of six triangles).

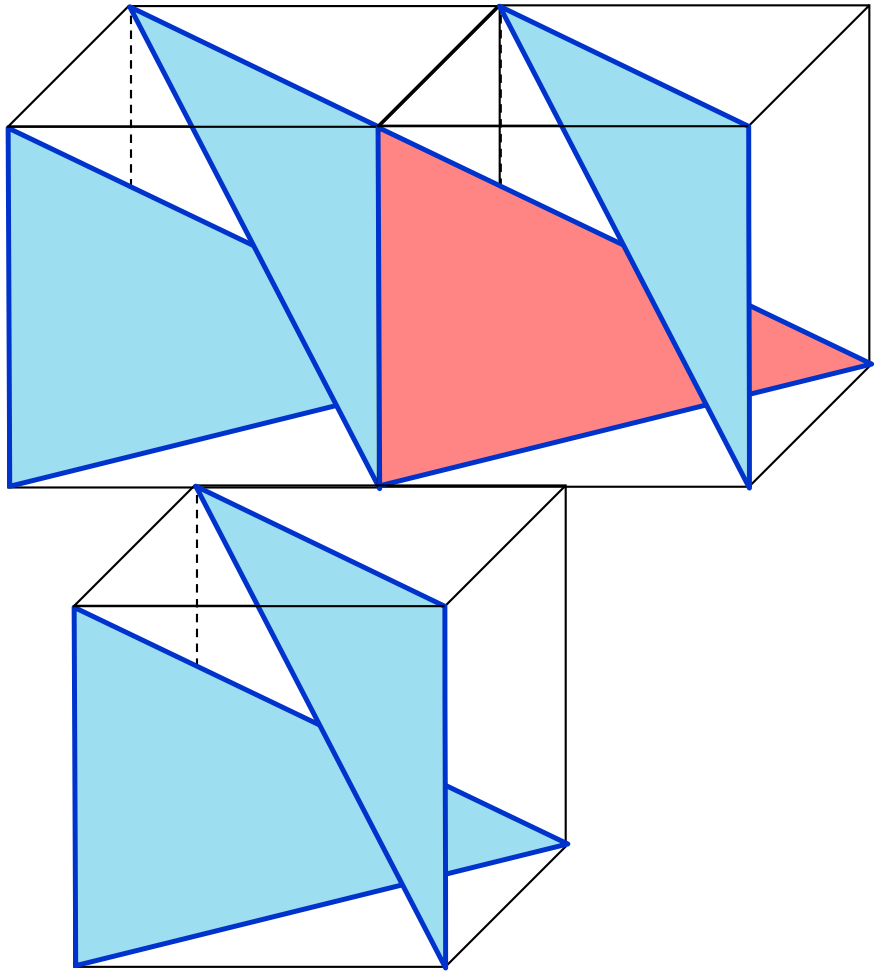


Figure 4.

Questions and problems

16. Exemplify the BSP Tree based visualization for the 3D object if the image in the orthogonal projection looks like in the figure.

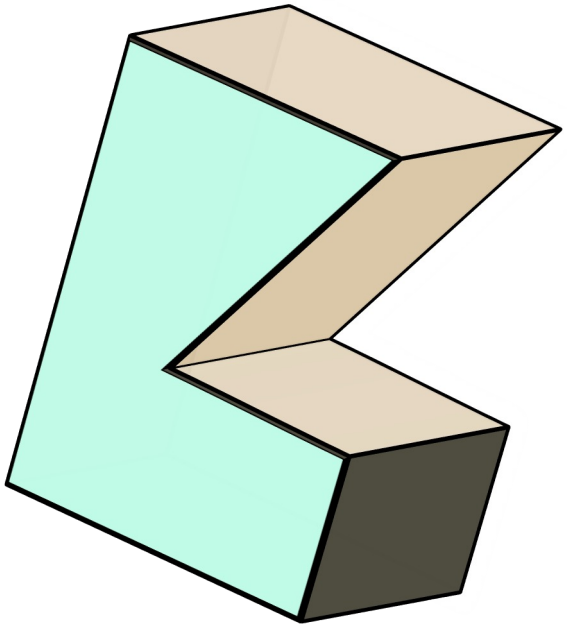


Figure 5.

Questions and problems

17. Exemplify the BSP Tree based visualization for the 3D object (composed of eight polygons), if the image in the orthogonal projection looks like in the figure.

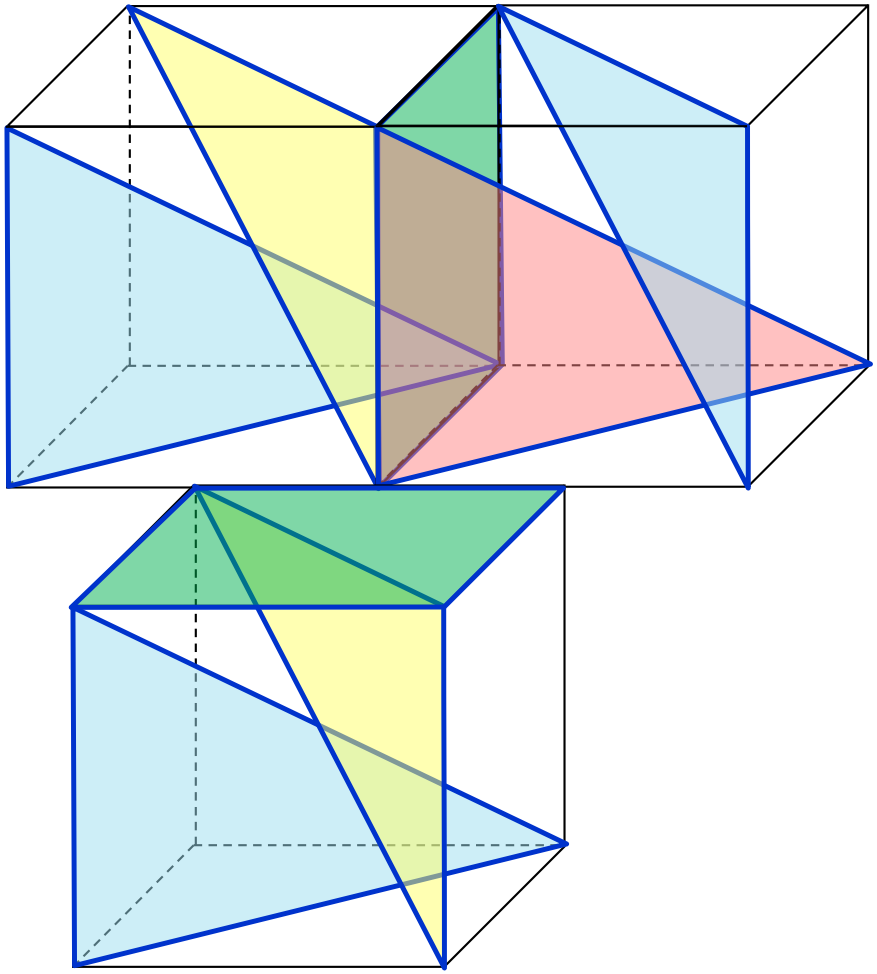


Figure 6.